

P12001 在小小的奶龙山里面挖呀挖呀挖 题解

注意到一条路径需要搞好关系的公司数，其实就是路径上所有点权中出现过的**不同质因子个数**。

比如路径上有 12, 15, 7，那么涉及的质数就是 2, 3, 5, 7，答案为 4。

所以问题变成：求树上一条路径上出现了多少种不同质因子。

考虑树上莫队。

先把每个 a_i 分解质因数。这里有个比较舒服的性质：

$$2 \times 3 \times 5 \times 7 \times 11 \times 13 = 30030 \leq 10^5,$$

而

$$2 \times 3 \times 5 \times 7 \times 11 \times 13 \times 17 > 10^5.$$

所以每个 a_i 至多只有 6 个不同质因子，直接开 `f[N][7]` 存就行，不需要 `set` 之类的东西。

接下来考虑怎么把树上路径变成莫队区间。

DFS 的时候每个点记录两次，第一次进入时记 tin_u ，退出时记 $tout_u$ ，得到长度为 $2n$ 的 Euler 序。

维护莫队区间时，每加入或删除一个位置，就把对应节点的状态翻转。一个点在区间中出现两次相当于没有出现，出现一次才算在当前路径里。

对于询问 (u, v) ，先保证 $tin_u < tin_v$ ，令 $w = LCA(u, v)$ 。

如果 $w = u$ ，说明 u 是 v 的祖先，直接取

$$[tin_u, tin_v].$$

否则取

$$[tout_u, tin_v].$$

此时区间对应的点集会少掉 LCA，所以处理这个询问时额外把 w 翻转一次，记录完答案再翻转回来。

这样就变成普通莫队了。

维护答案也很直接。设 `c[p]` 表示当前点集中有多少个点包含质因子 p ，`now` 表示当前存在多少种质因子。

加入一个节点时枚举它的所有质因子，如果原来 `c[p]==0` 就让 `now++`；删除时如果 `--c[p]==0` 就让 `now--`。

由于一个点最多只有 6 个不同质因子，所以 `change` 的常数很小。题目还特意提醒常数，这里质数直接拿来当数组下标，不用离散化，也尽量别上 `map`、`set`。

莫队排序正常做奇偶块优化即可。块长我这里取 $2n/\sqrt{q}$ 。

```
#include <bits/stdc++.h>
using namespace std;
```

```
namespace TANGYIXIAO {

const int N=5e4+5,A=1e5+5,L=17;

int n,q,a[N];
int mn[A],f[N][7],fc[N];
vector<int> g[N];

int dep[N],fa[N][L];
int tin[N],tout[N],e[N<<1],tim;

int vis[N],c[A],ans[N],now,B;

struct Node {
    int l,r,id,w,b;
} qu[N];

inline void init() {
    for (int i=2;i<A;i++) {
        if (!mn[i]) {
            mn[i]=i;
            if (1LL*i*i<A) {
                for (int j=i*i;j<A;j+=i) {
                    if (!mn[j]) {
                        mn[j]=i;
                    }
                }
            }
        }
    }
    return;
}

inline void dfs(int u,int p) {
    dep[u]=dep[p]+1;
    fa[u][0]=p;
    for (int j=1;j<L;j++) {
        fa[u][j]=fa[fa[u][j-1]][j-1];
    }

    tin[u]=++tim;
    e[tim]=u;

    for (int v:g[u]) {
        if (v!=p) {
            dfs(v,u);
        }
    }

    tout[u]=++tim;
    e[tim]=u;
    return;
}
```

```

inline int lca(int x,int y) {
    if (dep[x]<dep[y]) {
        swap(x,y);
    }

    int d=dep[x]-dep[y];
    for (int j=0;j<L;j++) {
        if ((d>>j)&1) {
            x=fa[x][j];
        }
    }

    if (x==y) {
        return x;
    }

    for (int j=L-1;j>=0;j--) {
        if (fa[x][j]!=fa[y][j]) {
            x=fa[x][j];
            y=fa[y][j];
        }
    }

    return fa[x][0];
}

inline void change(int x) {
    if (!vis[x]) {
        vis[x]=1;
        for (int j=0;j<fc[x];j++) {
            int p=f[x][j];
            if (!c[p]) {
                now++;
            }
            c[p]++;
        }
    } else {
        vis[x]=0;
        for (int j=0;j<fc[x];j++) {
            int p=f[x][j];
            c[p]--;
            if (!c[p]) {
                now--;
            }
        }
    }
    return;
}

inline bool cmp(const Node &x,const Node &y) {
    if (x.b!=y.b) {
        return x.b<y.b;
    }
}

```

```
    if (x.b&1) {
        return x.r>y.r;
    }
    return x.r<y.r;
}

inline void solve(int Task_Id) {
    cin>>n>>q;

    init();

    for (int i=1;i<=n;i++) {
        cin>>a[i];
        int x=a[i];
        while (x>1) {
            int p=mn[x];
            f[i][fc[i]++]=p;
            while (x%p==0) {
                x/=p;
            }
        }
    }

    for (int i=1;i<n;i++) {
        int u,v;
        cin>>u>>v;
        g[u].push_back(v);
        g[v].push_back(u);
    }

    dfs(1,0);

    B=max(1,(int)(tim/sqrt((double)q)));

    for (int i=1;i<=q;i++) {
        int u,v;
        cin>>u>>v;

        if (tin[u]>tin[v]) {
            swap(u,v);
        }

        int w=lca(u,v);

        if (w==u) {
            qu[i]={tin[u],tin[v],i,0,(tin[u]-1)/B};
        } else {
            qu[i]={tout[u],tin[v],i,w,(tout[u]-1)/B};
        }
    }

    sort(qu+1,qu+q+1,cmp);

    int l=1,r=0;
```

```

    for (int i=1;i<=q;i++) {
        while (l>qu[i].l) {
            change(e[--l]);
        }
        while (r<qu[i].r) {
            change(e[++r]);
        }
        while (l<qu[i].l) {
            change(e[l++]);
        }
        while (r>qu[i].r) {
            change(e[r--]);
        }

        if (qu[i].w) {
            change(qu[i].w);
        }

        ans[qu[i].id]=now;

        if (qu[i].w) {
            change(qu[i].w);
        }
    }

    for (int i=1;i<=q;i++) {
        cout<<ans[i]<<'\\n';
    }

    return;
}

} // namespace TANGYIXIAO

signed main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    TANGYIXIAO::solve(1);

    return 0;
}

```

整体就是质因数分解 + 树上莫队，复杂度足够过 $n, q \leq 5 \times 10^4$ 。

P10779 BZOJ4316 小 C 的独立集 题解

注意到题目保证每条边属于且仅属于一个简单环，所以这张图就是仙人掌，而且不会有桥。

这种图直接在原图上做独立集 DP 不太处理好环，考虑建圆方树。用 Tarjan 求点双，对于每个点双新建一个方点，把这个点双里的所有原点连向它。因为这里每个点双一定就是一个简单环，所以建出来的是一片森林。

设原点 u 的 DP 为 $f_{u,0}, f_{u,1}$ ，分别表示不选和选择 u 时，圆方树子树内的最大独立集。

对于一个原点 u ，如果它下面连着若干个环，那么这些环之间已经互不影响，直接加起来：

$$f_{u,0} = \sum f_{v,0},$$

$$f_{u,1} = 1 + \sum f_{v,1},$$

其中 v 是 u 的儿子方点。这里方点的 $f_{v,s}$ 表示：它的父亲原点状态为 s 时，这个环以及下面部分的最优答案，且不重复计算父亲本身。

于是只剩下一个环怎么转移。

设当前方点对应的环按顺序是

$$p, v_1, v_2, \dots, v_k,$$

其中 p 是它在圆方树上的父亲。固定 p 选或者不选之后，剩下的部分本质上就是一条链做独立集 DP，只需要注意最后的 v_k 和 p 之间还有一条边。

代码里用 x_0, x_1 表示当前走到的最后一个点不选或选择时的答案。转移就是普通链上独立集：

$$y_0 = \max(x_0, x_1) + f_{v,0},$$

$$y_1 = x_0 + f_{v,1}.$$

如果 p 被选，那么最后一个点一定不能选；否则两种状态取最大值即可。

还有一个实现上的问题：Tarjan 找到一个点双后，我们需要知道这个环上点的顺序。直接把这个点双里的边弹出来，因为它一定是简单环，所以每个点在当前点双内度数都是 2，临时记录每个点的两个邻点，随便找一个点一路走就能还原整个环。

图不一定连通，所以最后对圆方树森林的每个连通块分别做一次 DP。

```
#include <bits/stdc++.h>
using namespace std;

namespace TANGYIXIAO {

const int N=5e4+5,M=6e4+5,T=12e4+5,INF=1e9;

int n,m,eu[M],ev[M];
vector<pair<int,int>> g[N];
vector<int> tr[T],cyc[T];

int dfn[N],low[N],tim,st[M],top,tot;
int deg[N],n1[N],n2[N];
int f[T][2],vis[T];

inline void build(vector<int> &e) {
```

```

    int o=++tot;
    vector<int> v;
    for (int id:e) {
        int x=eu[id],y=ev[id];
        if (!deg[x]) {
            v.push_back(x);
        }
        if (!deg[y]) {
            v.push_back(y);
        }
        if (!deg[x]) {
            n1[x]=y;
        } else {
            n2[x]=y;
        }
        if (!deg[y]) {
            n1[y]=x;
        } else {
            n2[y]=x;
        }
        deg[x]++;
        deg[y]++;
    }
    int s=v[0],p=0,u=s;
    for (;;) {
        cyc[o].push_back(u);
        int x=(n1[u]!=p?n1[u]:n2[u]);
        p=u;
        u=x;
        if (u==s) {
            break;
        }
    }
    for (int x:v) {
        tr[o].push_back(x);
        tr[x].push_back(o);
        deg[x]=0;
    }
    return;
}

inline void tarjan(int u,int pe) {
    dfn[u]=low[u]=++tim;
    for (auto [v,id]:g[u]) {
        if (id==pe) {
            continue;
        }
        if (!dfn[v]) {
            st[++top]=id;
            tarjan(v,id);
            low[u]=min(low[u],low[v]);
            if (low[v]>=dfn[u]) {
                vector<int> e;
                for (;;) {

```

```

        int x=st[top--];
        e.push_back(x);
        if (x==id) {
            break;
        }
    }
    build(e);
}
} else if (dfn[v]<dfn[u]) {
    st[++top]=id;
    low[u]=min(low[u],dfn[v]);
}
}
return;
}

inline void dfs(int u,int p) {
    vis[u]=1;
    if (u<=n) {
        f[u][0]=0;
        f[u][1]=1;
        for (int v:tr[u]) {
            if (v==p) {
                continue;
            }
            dfs(v,u);
            f[u][0]+=f[v][0];
            f[u][1]+=f[v][1];
        }
        return;
    }
    for (int v:tr[u]) {
        if (v!=p) {
            dfs(v,u);
        }
    }
    int k=cyc[u].size(),pos=0;
    for (int i=0;i<k;i++) {
        if (cyc[u][i]==p) {
            pos=i;
            break;
        }
    }
    for (int s=0;s<=1;s++) {
        int x0=(s?-INF:0),x1=(s?0:-INF);
        for (int i=1;i<k;i++) {
            int v=cyc[u][(pos+i)%k];
            int y0=max(x0,x1)+f[v][0];
            int y1=x0+f[v][1];
            x0=y0;
            x1=y1;
        }
        f[u][s]=(s?x0:max(x0,x1));
    }
}

```



```
        return;
    }

    inline void solve(int Task_Id) {
        cin>>n>>m;
        tot=n;
        for (int i=1;i<=m;i++) {
            int u,v;
            cin>>u>>v;
            eu[i]=u;
            ev[i]=v;
            g[u].push_back({v,i});
            g[v].push_back({u,i});
        }
        for (int i=1;i<=n;i++) {
            if (!dfn[i]) {
                tarjan(i,0);
                top=0;
            }
        }
        int ans=0;
        for (int i=1;i<=n;i++) {
            if (!vis[i]) {
                dfs(i,0);
                ans+=max(f[i][0],f[i][1]);
            }
        }
        cout<<ans<<'\n';
        return;
    }

} // namespace TANGYIXIAO

signed main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    TANGYIXIAO::solve(1);

    return 0;
}
```

P8819 [CSP-S 2022] 星战

题解链接: [洛谷题解区随机哈希实现](#)

思路

注意到题目要求每个点恰好只有一条可用出边。只要这个条件成立，从任意点开始沿唯一出边一直走，因为点数有限，最后一定会走进一个环，所以题目的第一个条件其实不用额外考虑。

记点 u 当前的可用出度为 d_u ，我们只需要判断

$$d_u = 1 \quad (1 \leq u \leq n).$$

考虑维护

$$E = \sum_{u=1}^n d_u$$

和

$$P = \sum_{u=1}^n \binom{d_u}{2}.$$

由于所有 d_u 都是非负整数，有

$$P = 0 \iff d_u \in \{0, 1\}.$$

所以

$$E = n, P = 0$$

当且仅当所有 $d_u = 1$ 。

这样就完全不用随机哈希了。

如果某个点的当前出度为 x ，那么：

- $x \rightarrow x + 1$ 时， P 增加 x ；
- $x \rightarrow x - 1$ 时， P 减少 $x - 1$ 。

所以单边修改很好处理。

对于终点 v ，记它在原图中的所有入边起点组成集合

$$S_v = \{u \mid (u, v) \in E_0\}.$$

设执行一次整列操作前

$$T_v = \sum_{u \in S_v} d_u.$$

如果 S_v 内所有点的出度同时 $+1$ ，那么

$$\Delta P = T_v.$$

如果同时 -1 ，那么

$$\Delta P = |S_v| - T_v.$$

于是问题就变成了：支持单点修改 $d_u \pm 1$ ，以及对一个静态集合 S_v 整体 ± 1 ，同时快速求出 T_v 。

这里有个坑。操作 2, 4 会直接修改一个终点的全部入边，但这些边之前可能已经被操作 1, 3 单独改过。

对每个终点 v 维护一个基准状态 $b_v \in \{0, 1\}$ ，以及例外集合 X_v ，表示当前状态和 b_v 不同的边。

初始显然有

$$b_v = 1, \quad X_v = \emptyset.$$

操作 1, 3 只改单条边。修改以后, 如果这条边的状态等于 b_v , 就从 X_v 删除, 否则加入 X_v 。

操作 2, 4 时, 先把 X_v 全部恢复到基准状态并清空。如果当前 b_v 和目标状态不同, 再对整个 S_v 做一次整体修改并更新 b_v 。

所有被清掉的例外都一定来自之前的一次单边操作, 所以这部分总共只会处理 $O(q)$ 条例外。代码里只给真正出现在 1, 3 操作中的边编号, 再用 swap-delete 维护即可。

取阈值 B 。

如果

$$|S_v| > B,$$

称 v 为重终点, 否则为轻终点。

设重终点数量为 H , 显然

$$H \leq \frac{m}{B}.$$

把对应集合编号为 $G_1, G_2, \dots, G_H$ 。

对于每个重集合维护懒标记 tag_h , 令

$$d_u = val_u + \sum_{h: u \in G_h} tag_h.$$

初始所有重集合对应的边都存在, 所以 $tag_h = 1$, 轻边的贡献直接放进 val_u 。

再维护

$$hs_h = \sum_{u \in G_h} val_u.$$

单点修改

修改点 u 时, 先通过

$$d_u = val_u + \sum_{h: u \in G_h} tag_h$$

求出修改前的真实出度, 然后更新 E, P 。

接着修改 val_u 。

由于 u 可能同时属于若干个重集合, 所以对于所有满足 $u \in G_h$ 的 h , 还要同步修改 hs_h 。

重集合修改

预处理

$$hh_{h,k} = |G_h \cap G_k|.$$

那么

$$\sum_{u \in G_h} d_u = hs_h + \sum_{k=1}^H hh_{h,k} tag_k.$$

所以重集合 G_h 整体修改时，不需要枚举集合中的点，直接利用上式求出 T_h ，更新 E, P ，最后修改对应的 tag_h 即可。

轻集合修改

轻集合大小至多为 B ，所以它的 val 部分直接枚举。

再记

$$c_{v,h} = |S_v \cap G_h|.$$

于是

$$T_v = \sum_{u \in S_v} val_u + \sum_{h=1}^H c_{v,h} tag_h.$$

整体修改时直接枚举 S_v 修改对应的 val_u ，并根据 $c_{v,h}$ 同步修改各个 hs_h 。

实际代码里只对真正出现过操作 2, 4 的轻终点建这部分信息。交比较少就用稀疏表示，交比较多就连续存储，这些都只是压常，不影响做法。

最后每次询问后判断

$$E = n \text{ 且 } P = 0$$

即可。

P10375 [AHOI2024 初中组/科大国创杯初中组 2024] 计数 题解

先考虑一个确定的糖果序列什么时候能吃完。

看当前最左边还没被吃掉的糖果。它最终被吃掉的时候，由于左边已经没有糖果了，所以它一定是某次操作的左端点，并且和右边某个同色糖果一起被吃掉。也就是说，整个序列可以从左往右划成若干段，每一段的首尾颜色相同。

于是对于一个确定的序列，设 ok_i 表示前 i 个糖果能不能吃完， $ok_0 = 1$ ，那么有

$$ok_i = 1 \iff \exists j < i, ok_{j-1} = 1 \text{ 且 } c_j = c_i.$$

直接按这个东西计数。

假设现在已经确定了前 i 个糖果，考虑集合

$$S_i = \{c_j \mid 1 \leq j \leq i, ok_{j-1} = 1\}.$$

也就是所有可以作为下一段左端点的颜色。

那么加入第 $i + 1$ 个糖果时：

- 如果它的颜色在 S_i 中，那么一定有 $ok_{i+1} = 1$ ；
- 否则一定有 $ok_{i+1} = 0$ 。

显然我们并不关心 S_i 里面具体有哪些颜色，只关心它有多少种颜色。

设 $f[j][0/1]$ 表示当前 $|S_i| = j$ ，并且 ok_i 为 0/1 的序列数量。

假设当前 $|S_i| = j$ 。

如果新糖果选了 S_i 中已有的颜色，一共有 j 种选法。此时新的前缀一定合法，而且集合大小不变，所以

$$g_{j,1} += (f_{j,0} + f_{j,1})j.$$

如果选了一个 S_i 外的颜色，一共有 $m - j$ 种选法，此时新的前缀一定不合法。

如果原来 $ok_i = 0$ ，那么第 $i + 1$ 个位置也不能作为下一段的开头，所以集合不变：

$$g_{j,0} += f_{j,0}(m - j).$$

如果原来 $ok_i = 1$ ，那么第 $i + 1$ 个位置可以作为下一段的开头。又因为它的颜色原本不在 S_i 中，所以集合大小加一：

$$g_{j+1,0} += f_{j,1}(m - j).$$

初始时还没有放任何糖果， S_0 为空，并且空前缀可以吃完，所以

$$f_{0,1} = 1.$$

做完 n 次转移后，把所有 $f[j][1]$ 加起来就是答案。

状态只有 $O(n)$ 个，滚动数组一下，总共 $O(n^2)$ ，对于 $n \leq 3000$ 很轻松。

```
#include <bits/stdc++.h>
using namespace std;

namespace TANGYIXIAO {

const int N=3e3+5;
const long long MOD=1e9+7;

int n;
long long m,f[2][N][2];

inline void solve(int Task_Id) {
    cin>>n>>m;

    f[0][0][1]=1;
```

```

for (int i=0;i<n;i++) {
    int x=i&1,y=x^1;
    memset(f[y],0,sizeof(f[y]));

    for (int j=0;j<=i;j++) {
        long long in=j;
        long long out=(j<=m?m-j:0);

        long long s=(f[x][j][0]+f[x][j][1])%MOD;

        f[y][j][1]=(f[y][j][1]+s*in)%MOD;
        f[y][j][0]=(f[y][j][0]+f[x][j][0]*out)%MOD;
        f[y][j+1][0]=(f[y][j+1][0]+f[x][j][1]*out)%MOD;
    }
}

long long ans=0;

for (int j=0;j<=n;j++) {
    ans=(ans+f[n&1][j][1])%MOD;
}

cout<<ans<<'\n';

return;
}

} // namespace TANGYIXIAO

signed main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    TANGYIXIAO::solve(1);

    return 0;
}

```

P3332 [ZJOI2013] K 大数查询 题解

注意到修改虽然是在一段位置里插入同一个数，但是询问本质还是一个**区间第 k 大**。

如果直接在线做，很容易想到权值线段树套区间线段树，不过这题可以直接整体二分，空间小很多。

思路

考虑当前正在处理的答案值域 $[L, R]$ ，令

$$mid = L + \left\lfloor \frac{R - L + 1}{2} \right\rfloor.$$

我们把值域拆成：

- 左边 $[L, mid - 1]$;
- 右边 $[mid, R]$ 。

因为求的是第 k 大，所以先考虑右边。

对于一条修改

```
1 l r c
```

如果 $c \geq mid$ ，那么这个数属于右半边。我们在线段树上把 $[l, r]$ 区间加一。

这样，对于一条询问

```
2 l r k
```

查询线段树中 $[l, r]$ 的区间和，设为 x 。

这里的 x 就表示：在当前询问发生以前， $[l, r]$ 中一共有多少个值不小于 mid 的数。

于是：

- 如果 $x \geq k$ ，第 k 大一定在右半边；
- 否则答案在左半边，并且右半边的 x 个数已经全部跳过，所以令

$$k \leftarrow k - x.$$

然后把所有操作按照去左边还是右边稳定地分开，继续递归即可。

这里一定要**保持操作原来的相对顺序**。因为一条询问只能看见它之前的修改，整体二分虽然会重新排列操作，但是同一个子问题内部的时间顺序不能乱。

每一层处理完成以后，别忘了把这一层在线段树上的修改撤销。

线段树维护什么

一次修改 `1 l r c`，如果当前认为它属于右半值域，就相当于：

$$[l, r] \leftarrow [l, r] + 1.$$

询问则需要求某个区间的和。

所以普通的**区间加、区间求和**和**线段树**就够了。

这里有一个比较阴的坑：

$$nm = 5 \times 10^4 \times 5 \times 10^4 = 2.5 \times 10^9,$$

已经超过 `int`。

而且题面直接给了询问中的 $c < 2^{63}$ ，因此线段树的 `sum`、`tag` 和询问的 k 全部开 `long long`。

另外值域有负数，别写一些依赖负数右移的奇怪 `mid`，直接写：

```
int mid=vl+(vr-vl+1)/2;
```

最省心。

复杂度为 $O(m \log n \log n)$ ，空间 $O(n + m)$ 。

代码

```
#include <bits/stdc++.h>
using namespace std;

namespace TANGYIXIAO {
    const int N=5e4+5;

    struct Node {
        int op,l,r,id;
        long long c;
    } a[N],b[N];

    int n,m,cnt,ans[N];
    long long tr[N<<2],tag[N<<2];
    bool go[N];

    void push(int p,int l,int r) {
        if (!tag[p] || l==r) {
            return;
        }
        int mid=(l+r)>>1;
        tr[p<<1]+=tag[p]*(mid-l+1);
        tr[p<<1|1]+=tag[p]*(r-mid);
        tag[p<<1]+=tag[p];
        tag[p<<1|1]+=tag[p];
        tag[p]=0;
    }

    void add(int p,int l,int r,int ql,int qr,long long v) {
        if (ql<=l && r<=qr) {
            tr[p]+=v*(r-l+1);
            tag[p]+=v;
            return;
        }
        push(p,l,r);
        int mid=(l+r)>>1;
        if (ql<=mid) {
            add(p<<1,l,mid,ql,qr,v);
        }
    }
}
```



```
        if (qr>mid) {
            add(p<<1|1,mid+1,r,ql,qr,v);
        }
        tr[p]=tr[p<<1]+tr[p<<1|1];
    }

    long long ask(int p,int l,int r,int ql,int qr) {
        if (ql<=l && r<=qr) {
            return tr[p];
        }
        push(p,l,r);
        int mid=(l+r)>>1;
        long long res=0;
        if (ql<=mid) {
            res+=ask(p<<1,l,mid,ql,qr);
        }
        if (qr>mid) {
            res+=ask(p<<1|1,mid+1,r,ql,qr);
        }
        return res;
    }

    void solve(int ql,int qr,int vl,int vr) {
        if (ql>qr) {
            return;
        }
        if (vl==vr) {
            for (int i=ql;i<=qr;i++) {
                if (a[i].op==2) {
                    ans[a[i].id]=vl;
                }
            }
            return;
        }

        int mid=vl+(vr-vl+1)/2;

        for (int i=ql;i<=qr;i++) {
            if (a[i].op==1) {
                if (a[i].c>=mid) {
                    add(1,1,n,a[i].l,a[i].r,1);
                    go[i]=1;
                } else {
                    go[i]=0;
                }
            } else {
                long long x=ask(1,1,n,a[i].l,a[i].r);
                if (x>=a[i].c) {
                    go[i]=1;
                } else {
                    a[i].c-=x;
                    go[i]=0;
                }
            }
        }
    }
```

```
    }

    for (int i=ql;i<=qr;i++) {
        if (a[i].op==1 && a[i].c>=mid) {
            add(1,1,n,a[i].l,a[i].r,-1);
        }
    }

    int p=ql;
    for (int i=ql;i<=qr;i++) {
        if (!go[i]) {
            b[p++]=a[i];
        }
    }
    int k=p-1;
    for (int i=ql;i<=qr;i++) {
        if (go[i]) {
            b[p++]=a[i];
        }
    }
    for (int i=ql;i<=qr;i++) {
        a[i]=b[i];
    }

    solve(ql,k,vl,mid-1);
    solve(k+1,qr,mid,vr);
}

}

signed main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    using namespace TANGYIXIAO;

    cin>>n>>m;
    for (int i=1;i<=m;i++) {
        cin>>a[i].op>>a[i].l>>a[i].r>>a[i].c;
        if (a[i].op==2) {
            a[i].id=++cnt;
        }
    }

    solve(1,m,-n,n);

    for (int i=1;i<=cnt;i++) {
        cout<<ans[i]<<'\\n';
    }

    return 0;
}
```

P4092 [HEOI2016/TJOI2016] 树 题解

比较裸的树剖。

要求的是 u 到根这条链上深度最大的已标记点。考虑树链剖分以后维护每个点是否被标记，在线段树的 dfn_u 位置存 dfn_u ，没被标记就存 0。

为什么可以直接取最大的 dfn ？因为对于祖先关系，一定有

$$u \text{ 是 } v \text{ 的祖先} \Rightarrow dfn_u < dfn_v.$$

而一次询问中考虑的所有点都在根到 u 的同一条路径上，所以其中 dfn 最大的已标记点就是深度最大的那个，也就是最近的已标记祖先。

于是询问时正常树剖跳链，把 $1 \rightarrow u$ 拆成 $O(\log n)$ 个连续区间，每个区间在线段树上查询最大值即可。

标记操作就是单点修改。重复标记没有影响，重新赋成 $dfn[u]$ 即可。

最后用 $rk[dfn]$ 找回节点编号。

复杂度就是单次修改 $O(\log n)$ ，单次询问 $O(\log^2 n)$ 。

```
#include <bits/stdc++.h>
using namespace std;

namespace TANGYIXIAO {

const int N=1e5+5;

int n,q;
vector<int> g[N];

int fa[N],dep[N],siz[N],son[N];
int top[N],dfn[N],rk[N],tim;
int tr[N<<2];

inline void dfs1(int u,int f) {
    fa[u]=f;
    dep[u]=dep[f]+1;
    siz[u]=1;
    for (int v:g[u]) {
        if (v==f) {
            continue;
        }
        dfs1(v,u);
        siz[u]+=siz[v];
        if (siz[v]>siz[son[u]]) {
            son[u]=v;
        }
    }
    return;
}
```

```
inline void dfs2(int u,int t) {
    top[u]=t;
    dfn[u]=++tim;
    rk[tim]=u;
    if (son[u]) {
        dfs2(son[u],t);
    }
    for (int v:g[u]) {
        if (v==fa[u]||v==son[u]) {
            continue;
        }
        dfs2(v,v);
    }
    return;
}

inline void change(int p,int l,int r,int x) {
    if (l==r) {
        tr[p]=l;
        return;
    }
    int mid=(l+r)>>1;
    if (x<=mid) {
        change(p<<1,l,mid,x);
    } else {
        change(p<<1|1,mid+1,r,x);
    }
    tr[p]=max(tr[p<<1],tr[p<<1|1]);
    return;
}

inline int query(int p,int l,int r,int x,int y) {
    if (x<=l&&r<=y) {
        return tr[p];
    }
    int mid=(l+r)>>1,res=0;
    if (x<=mid) {
        res=max(res,query(p<<1,l,mid,x,y));
    }
    if (y>mid) {
        res=max(res,query(p<<1|1,mid+1,r,x,y));
    }
    return res;
}

inline int ask(int u) {
    int res=0;
    while (top[u]!=1) {
        res=max(res,query(1,1,n,dfn[top[u]],dfn[u]));
        u=fa[top[u]];
    }
    res=max(res,query(1,1,n,dfn[1],dfn[u]));
    return rk[res];
}
```

```

}

inline void solve(int Task_Id) {
    cin>>n>>q;
    for (int i=1;i<n;i++) {
        int u,v;
        cin>>u>>v;
        g[u].push_back(v);
        g[v].push_back(u);
    }

    dfs1(1,0);
    dfs2(1,1);

    change(1,1,n,dfn[1]);

    for (int i=1;i<=q;i++) {
        char op;
        int x;
        cin>>op>>x;
        if (op=='C') {
            change(1,1,n,dfn[x]);
        } else {
            cout<<ask(x)<<'\n';
        }
    }

    return;
}

} // namespace TANGYIXIAO

signed main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    TANGYIXIAO::solve(1);

    return 0;
}

```

P4052 [JSOI2007] 文本生成器 题解

正着统计“至少包含一个单词”的文章不太方便，考虑反过来：

答案 = 26^m - 一个单词都不包含的文章数.

于是问题就变成了：有多少个长度为 m 的字符串，不包含给出的任何一个模式串。

显然可以上 AC 自动机。

把所有单词插进 Trie，然后建 fail。对于一个节点，如果它本身是某个单词的结尾，或者它的 fail 指向的状态已经包含某个单词，那么只要走到这个节点，当前字符串就已经出现了至少一个给定单词。

记 $\text{bad}[u]$ 表示状态 u 是否已经匹配到某个单词，建 fail 时直接做

```
bad[u] |= bad[fail[u]];
```

即可。

然后 DP。

设 $f[i][u]$ 表示已经填了 i 个字符，现在 AC 自动机停在状态 u ，并且到目前为止还没有出现任何给定单词的方案数。

枚举下一个字符 c ，设

$$v = \text{tr}[u][c].$$

如果 $\text{bad}[v]$ ，这种方案显然不能要；否则转移

$$f_{i+1,v} += f_{i,u}.$$

最后把所有状态的 $f_{m,u}$ 加起来，就是完全不可读的文章数量，再用 26^m 减掉即可。

状态数最多是所有模式串长度之和，也就 6000 左右，直接跑就行。

```
#include <bits/stdc++.h>
using namespace std;

namespace TANGYIXIAO {

const int N=6e3+5,MOD=1e4+7;

int n,m,tot;
int tr[N][26],fail[N],bad[N];
int f[2][N];

inline void insert(string s) {
    int u=0;
    for (char c:s) {
        int x=c-'A';
        if (!tr[u][x]) {
            tr[u][x]=++tot;
        }
        u=tr[u][x];
    }
    bad[u]=1;
    return;
}

inline void build() {
```

```

    queue<int> q;
    for (int c=0;c<26;c++) {
        if (tr[0][c]) {
            q.push(tr[0][c]);
        }
    }
    while (!q.empty()) {
        int u=q.front();
        q.pop();
        bad[u]|=bad[fail[u]];
        for (int c=0;c<26;c++) {
            if (tr[u][c]) {
                fail[tr[u][c]]=tr[fail[u]][c];
                q.push(tr[u][c]);
            } else {
                tr[u][c]=tr[fail[u]][c];
            }
        }
    }
    return;
}

inline void solve(int Task_Id) {
    cin>>n>>m;
    for (int i=1;i<=n;i++) {
        string s;
        cin>>s;
        insert(s);
    }

    build();

    f[0][0]=1;
    for (int i=0;i<m;i++) {
        int x=i&1,y=x^1;
        memset(f[y],0,sizeof(f[y]));
        for (int u=0;u<=tot;u++) {
            if (!f[x][u]) {
                continue;
            }
            for (int c=0;c<26;c++) {
                int v=tr[u][c];
                if (!bad[v]) {
                    f[y][v]=(f[y][v]+f[x][u])%MOD;
                }
            }
        }
    }

    int no=0;
    for (int u=0;u<=tot;u++) {
        no=(no+f[m&1][u])%MOD;
    }
}

```

```
int all=1;
for (int i=1;i<=m;i++) {
    all=all*26%MOD;
}

cout<<(all-no+MOD)%MOD<<'\\n';

return;
}

} // namespace TANGYIXIAO

signed main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    TANGYIXIAO::solve(1);

    return 0;
}
```

P1235 [CERC/SWERC2002] 血缘关系

这题递推不难，真正麻烦的是题目要求：

有多少精度就输出多少，而且必须准确。

所以不能直接用 `double`。

不过也没必要上高精度库，因为这题出现的概率分母始终都是 2 的幂，可以自己写一个非常简单的高精度整数。

相似度递推

设 $f_{x,y}$ 表示妖怪 x, y 的基因相似程度。

首先有

$$f_{x,x} = 1.$$

对于两个没有父母且不同的妖怪，题目保证它们毫无关系，所以

$$f_{x,y} = 0.$$

如果妖怪 x 的父母为 a, b ，那么 x 的一个基因：

- 有 $\frac{1}{2}$ 概率来自 a ；
- 有 $\frac{1}{2}$ 概率来自 b 。

因此对于任意妖怪 y ，

$$f_{x,y} = \frac{f_{a,y} + f_{b,y}}{2}.$$

所以先把家谱拓扑排序。

按照拓扑序枚举妖怪 x ，它的父母一定已经计算完成。再枚举拓扑序中位于它前面的妖怪 y ，直接转移：

$$f_{x,y} = f_{y,x} = \frac{f_{a,y} + f_{b,y}}{2}.$$

$n \leq 300$ ，直接存整个 f 数组即可。

怎么保证精确

连续经过若干代以后，相似度一定可以写成

$$\frac{p}{2^d},$$

其中 $d \leq n$ 。

注意

$$10^n = 2^n 5^n,$$

所以对于任意 $d \leq n$,

$$\frac{p}{2^d} \times 10^n$$

一定是整数。

于是我们干脆把所有概率统一扩大 10^n 倍。

也就是说实际存储

$$F_{x,y} = f_{x,y} \times 10^n.$$

那么：

$$F_{x,x} = 10^n,$$

转移仍然只是

$$F_{x,y} = \frac{F_{a,y} + F_{b,y}}{2}.$$

整个过程中不存在任何小数，也就没有精度误差。

10^{300} 普通整数显然存不下，不过我们只需要：

- 高精度加法；
- 高精度除以 2；
- 高精度乘一个小整数；
- 转字符串。

直接用数组以 10^9 为一位即可。

10^{300} 只有大约 34 个这样的块，开 40 位已经绰绰有余。

输出

现在存储的是

$$F = f \times 10^n.$$

题目要求百分数，所以需要输出

$$100f = \frac{100F}{10^n}.$$

因此把高精度整数乘 100，转成十进制字符串，然后从右往左数 n 位插入小数点即可。

最后删掉末尾多余的 0。

例如：

```
81.250000% -> 81.25%
50.000000% -> 50%
0.100000% -> 0.1%
```

代码

```
#include <bits/stdc++.h>
using namespace std;

namespace TANGYIXIAO {
    const int N=305,K=40;
    const long long B=100000000LL;

    struct Big {
        int a[K];

        Big() {
            memset(a,0,sizeof(a));
        }
    };

    int n,k,m;
    int fa[N][2],deg[N];
    int e[N][N],ec[N];
    int q[N],p[N],tot;

    Big f[N][N],S;

    void avg(Big &c,const Big &a,const Big &b) {
        long long v=0;

        for (int i=0;i<K;i++) {
```

```
        v+=1LL*a.a[i]+b.a[i];
        c.a[i]=v%B;
        v/=B;
    }

    v=0;
    for (int i=K-1;i>=0;i--) {
        long long x=v*B+c.a[i];
        c.a[i]=x/2;
        v=x&1;
    }
}

void mul(Big &a,int x) {
    long long v=0;

    for (int i=0;i<K;i++) {
        v+=1LL*a.a[i]*x;
        a.a[i]=v%B;
        v/=B;
    }
}

string str(const Big &a) {
    int p=K-1;

    for (;p>0 && !a.a[p];p--) {}

    string s=to_string(a.a[p]);

    for (int i=p-1;i>=0;i--) {
        string t=to_string(a.a[i]);
        s+=string(9-t.size(),'0')+t;
    }

    return s;
}

void print(const Big &a) {
    Big x=a;
    mul(x,100);

    string s=str(x);

    if ((int)s.size()<=n) {
        s=string(n+1-s.size(),'0')+s;
    }

    s.insert(s.end()-n,'.');

    for (;!s.empty() && s.back()=='0';) {
        s.pop_back();
    }
}
```

```
        if (!s.empty() && s.back()=='.') {
            s.pop_back();
        }

        cout<<s<<"%\n";
    }
}

signed main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    using namespace TANGYIXIAO;

    cin>>n>>k;

    for (int i=1;i<=k;i++) {
        int a,b,c;
        cin>>a>>b>>c;

        fa[a][0]=b;
        fa[a][1]=c;

        e[b][++ec[b]]=a;
        e[c][++ec[c]]=a;

        deg[a]+=2;
    }

    int l=1,r=0;

    for (int i=1;i<=n;i++) {
        if (!deg[i]) {
            q[++r]=i;
        }
    }

    for (;l<=r;l++) {
        int x=q[l];
        p[++tot]=x;

        for (int i=1;i<=ec[x];i++) {
            int y=e[x][i];
            deg[y]--;

            if (!deg[y]) {
                q[++r]=y;
            }
        }
    }

    int x=1;

    for (int i=1;i<=n%9;i++) {
```

```

        x*=10;
    }

    S.a[n/9]=x;

    for (int i=1;i<=n;i++) {
        int x=p[i];

        f[x][x]=S;

        if (!fa[x][0]) {
            continue;
        }

        int a=fa[x][0],b=fa[x][1];

        for (int j=1;j<i;j++) {
            int y=p[j];

            avg(f[x][y],f[a][y],f[b][y]);
            f[y][x]=f[x][y];
        }
    }

    cin>>m;

    for (int i=1;i<=m;i++) {
        int x,y;
        cin>>x>>y;

        print(f[x][y]);
    }

    return 0;
}

```

这里手写高精度其实很轻，因为根本不需要实现完整的高精度乘除法。

复杂度大概就是 $O(n^2K + mK)$ ，其中 $K = 40$ 是常数，完全够用。

P1155 [NOIP 2008 提高组] 双栈排序

注意到这题并不是直接模拟两个栈。

关键是先判断哪些数绝对不能进入同一个栈，然后就很容易想到二分图染色。

冲突关系

考虑两个位置 $i < j$ 。

什么时候 a_i, a_j 一定不能进入同一个栈?

假设后面还存在一个位置 $k > j$, 满足

$$a_k < a_i < a_j.$$

由于 $a_k < a_i$, 在读到 a_k 以前, a_i 肯定不能出栈, 否则输出序列中 a_i 会出现在更小的 a_k 前面。

所以读到 a_j 时, a_i 还在栈里。

但是又有

$$a_i < a_j.$$

如果把 a_j 压到 a_i 上面, 那么之后 a_i 应该比 a_j 先弹出, 却被 a_j 压住了, 显然寄了。

所以只要存在

$$i < j < k, \quad a_k < a_i < a_j,$$

那么 i, j 就不能进入同一个栈。

于是把每个位置看成一个点, 对于所有满足上述条件的 (i, j) 连边。

问题就变成了:

能不能把所有点分成两组, 使每条边两端属于不同组。

直接二分图染色。

如果不是二分图, 无解。

怎么快速判断有没有 k

暴力枚举 i, j, k 是 $O(n^3)$, 虽然 $n = 1000$, 但显然没必要。

预处理后缀最小值

$$mn_i = \min_{j=i}^n a_j.$$

那么对于 $i < j$, 存在 $k > j$ 满足

$$a_k < a_i$$

等价于

$$mn_{j+1} < a_i.$$

所以连边条件就是

$$a_i < a_j \quad \text{且} \quad mn_{j+1} < a_i.$$

这样 $O(n^2)$ 就能把图建出来。

字典序最小的染色

操作的字典序为

$$a < b < c < d.$$

所以显然能放进第一个栈就比放进第二个栈优。

一个二分图连通块的染色只有两种，本质上就是把两种颜色全部交换。

因此按照原序列从前往后枚举，每遇到一个还没有染色的连通块，就把当前这个点染成第一个栈。

这样这个连通块第一次影响答案时，得到的是操作 **a** 而不是 **c**，一定更优。

于是：

- 颜色 0：进入 S_1 ；
- 颜色 1：进入 S_2 。

怎么模拟

这里有个坑。

不能写成：

一个数入栈后，只要当前能弹出就一直弹。

因为字典序要求

$$a < b < c < d.$$

比如现在虽然第二个栈可以执行 **d**，但如果接下来还可以执行 **a**，显然应该先执行 **a**。

所以需要按照字典序贪心。

设当前应该输出的数为 now 。

当前数进入 S_1

a 是最小的操作，因此只要能直接压栈，就应该立即压。

但是如果当前

$$top(S_1) < a_i,$$

那么把 a_i 压进去以后，会把一个更小的数压住，显然不行。

因此必须先弹栈，直到

S_1 为空

或者

$$top(S_1) > a_i.$$

当然我们只能弹出当前应该输出的 now 。

如果 S_1 的栈顶不能弹，那么就只能尝试从 S_2 弹。

处理完以后执行 a 。

当前数进入 S_2

这时准备进行的是操作 c 。

由于

$$b < c,$$

所以在执行 c 以前，只要 S_1 能执行 b ，就一定先执行 b 。

然后同样判断 S_2 的栈顶。

如果

$$top(S_2) < a_i,$$

就必须先把挡住的位置处理掉。

处理过程中可能又使 S_1 可以执行 b ，所以在真正执行 c 以前，再把所有能执行的 b 全部执行掉。

最后把 a_i 压入 S_2 。

所有数都读入以后，再按照 $1, 2, \dots, n$ 的顺序把两个栈清空即可。

这里千万别把 d 随便提前，因为 d 是四种操作中字典序最大的。

代码

```
#include <bits/stdc++.h>
using namespace std;

namespace TANGYIXIAO {
    const int N=1e3+5;

    int n,a[N],mn[N],col[N];
    int s1[N],s2[N],t1,t2,now=1,pos=1,cnt;
    bool g[N][N];
    char ans[N<<1];

    bool dfs(int x,int c) {
        col[x]=c;
        for (int i=1;i<=n;i++) {
            if (!g[x][i]) {
                continue;
            }
            if (col[i]==-1) {
                if (!dfs(i,c^1)) {
                    return false;
                }
            } else if (col[i]==c) {
                return false;
            }
        }
        return true;
    }
}
```



```
    }
    }
    return true;
}

signed main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    using namespace TANGYIXIAO;

    cin>>n;
    for (int i=1;i<=n;i++) {
        cin>>a[i];
    }

    mn[n+1]=n+1;
    for (int i=n;i>=1;i--) {
        mn[i]=min(mn[i+1],a[i]);
    }

    for (int i=1;i<=n;i++) {
        for (int j=i+1;j<=n;j++) {
            if (a[i]<a[j] && mn[j+1]<a[i]) {
                g[i][j]=g[j][i]=true;
            }
        }
    }

    memset(col,-1,sizeof(col));
    for (int i=1;i<=n;i++) {
        if (col[i]!=-1) {
            continue;
        }
        if (!dfs(i,0)) {
            cout<<0<<'\n';
            return 0;
        }
    }

    for (;cnt<2*n;) {
        if (pos<=n && !col[pos] && (!t1 || s1[t1]>a[pos])) {
            s1[++t1]=a[pos++];
            ans[++cnt]='a';
        } else if (t1 && s1[t1]==now) {
            t1--;
            now++;
            ans[++cnt]='b';
        } else if (pos<=n && col[pos] && (!t2 || s2[t2]>a[pos])) {
            s2[++t2]=a[pos++];
            ans[++cnt]='c';
        } else if (t2 && s2[t2]==now) {
            t2--;
        }
    }
}
```

```

        now++;
        ans[++cnt]='d';
    } else {
        cout<<0<<'\n';
        return 0;
    }
}

for (int i=1;i<=cnt;i++) {
    cout<<ans[i];
    if (i<cnt) {
        cout<<' ';
    }
}
cout<<'\n';

return 0;
}

```

建冲突图和染色都是 $O(n^2)$ ，最后模拟是 $O(n)$ 。

空间主要是邻接矩阵，为 $O(n^2)$ 。对于 $n \leq 1000$ 完全没问题。

B4229 [常州市赛 2024] 棋盘

原题目链接：[B4229 棋盘](#)

题解链接：[洛谷专栏 xgy54prw](#)

代码解释

```

#include<iostream>
using namespace std;

long long qi[3][150004],p[3],n,q,op,a;
// qi[1] 维护每一行第一个格子的颜色，qi[2] 维护每一列第一个格子的颜色。
// p[1]、p[2] 分别表示行方向、列方向上的颜色段数量。
// 颜色段一横一竖交叉出来的就是一个矩形连通块。

int main(){
    cin>>n>>q;

    for(int i=1;i<=2;i++)
        for(int j=1;j<=n;j++)
            qi[i][j]=j%2;
    // 初始棋盘黑白相间，所以行首和列首的颜色都是 1,0,1,0... 交替。

    qi[1][0]=qi[2][0]=qi[1][n+1]=qi[2][n+1]=-1;
    // 两侧塞哨兵，后面判断边界时不用再单独写 a==1 或 a==n。

```

```

p[1]=p[2]=n;
// 一开始颜色完全交替，每个位置自己就是一个颜色段，所以都是 n 段。

while(q--){
    cin>>op>>a;
    // op=1 翻一行，op=2 翻一列；只会改 qi[op] 这一条一维序列。

    if((qi[op][a]==1&&qi[op][a+1]==0&&qi[op][a-1]==0)||
        (qi[op][a]==0&&qi[op][a+1]==1&&qi[op][a-1]==1))p[op]-=2;
    // 形如 0 1 0 或 1 0 1。
    // 中间这个位置原来单独成一段，翻色后会把左右两段一起连起来，三段并成一段，所以减 2。

    else if((qi[op][a]==1&&qi[op][a+1]==0&&qi[op][a-1]==-1)||
        (qi[op][a]==1&&qi[op][a+1]==-1&&qi[op][a-1]==0)||
        (qi[op][a]==0&&qi[op][a+1]==1&&qi[op][a-1]==-1)||
        (qi[op][a]==0&&qi[op][a+1]==-1&&qi[op][a-1]==1))p[op]-=1;
    // 当前位置在边界，而且和唯一的邻居颜色不同。
    // 翻完以后两段并成一段，所以颜色段数减 1。

    else if((qi[op][a]==1&&qi[op][a+1]==1&&qi[op][a-1]==-1)||
        (qi[op][a]==1&&qi[op][a+1]==-1&&qi[op][a-1]==1)||
        (qi[op][a]==0&&qi[op][a+1]==-1&&qi[op][a-1]==0)||
        (qi[op][a]==0&&qi[op][a+1]==0&&qi[op][a-1]==-1))p[op]+=1;
    // 当前位置在边界，而且和唯一邻居颜色相同。
    // 翻完后它自己从原颜色段里裂出来，所以多一段。

    else if(qi[op][a]==qi[op][a-1]&&qi[op][a-1]==qi[op][a+1])p[op]+=2;
    // 三个位置同色，例如 1 1 1。
    // 中间翻完变成 1 0 1，一段被拆成三段，所以加 2。
    // 剩下没进这些分支的情况本来就在两段交界处，翻完也还是两段，p 不变。

    if(qi[op][a]==0)qi[op][a]=1;
    else qi[op][a]=0;
    // 前面只是更新颜色段数量，这里才真正把这一行/列的代表颜色翻掉。

    cout<<p[1]*p[2]<<endl;
    // 行颜色段数 × 列颜色段数，就是棋盘上的矩形同色连通块数量。
}

return 0;
}

```

P4201 [NOI2008] 设计路线

原题目链接：[P4201 设计路线](#)

题解链接：[洛谷专栏 cyyprbls](#)

代码解释

```

#include <cmath>
#include <cstdio>
#include <cstring>
#include <algorithm>

inline int read() {
    char c = getchar();
    int x = 0;

    while (c < '0' || c > '9') {
        c = getchar();
    }

    while (c >= '0' && c <= '9') {
        x = (x << 1) + (x << 3) + (c & 15);
        // x*10+(c-'0'), 这里把十进制位移和加法合并成一条表达式。
        c = getchar();
    }

    return x;
}

const int maxN = 100005, maxR = 12;

int n, m, p, f[3][maxN], g[3][maxR][maxN];
// f[i][u]: u 往儿子方向恰好用了 i 个铁路端点时, 子树最大不便利值的最优值。
// 代码里整体把真实值 +1, 避免后面 g 出现负下标。
// g[i][j][u]: u 往儿子方向恰好用了 i 个铁路端点, 且不便利值限制为 j 时的方案数。
// j 的范围很小, 所以第二维只开到 12。

struct List {
    int len, fst[maxN], nxt[maxN << 1], to[maxN << 1];

    List() {
        memset(fst, -1, sizeof(fst));
        // -1 表示邻接表到头。
    }

    inline void insert(int u, int v) {
        nxt[len] = fst[u];
        // 新边接到原来的表头前面。

        to[len] = v;
        // 这条边指向 v。

        fst[u] = len++;
        // u 的表头改成新边。
    }

    inline void link(int u, int v) {
        insert(u, v);
        insert(v, u);
    }
}

```

```

        // 原图是无向树，所以两个方向都存。
    }
} e;

inline int add(int x, int y) {
    x += y;
    return x >= p ? x - p : x;
    // 两个数本来都已经模 p，所以加完最多只需要减一次 p。
}

inline int mul(int x, int y) {
    return (long long) x * y % p;
    // 先转 long long，避免 int 乘法炸掉。
}

void dfs1(int u, int fa) {
    f[0][u] = 1;
    // 代码中的 1 对应真实不便值 0。

    f[1][u] = f[2][u] = 1e9;
    // 还没处理儿子时，不可能已经恰好用了 1/2 条铁路边，先放无穷大。

    for (int i = e.fst[u], v; ~i; i = e.nxt[i]) {
        v = e.to[i];

        if (v == fa) {
            continue;
            // 无向树里别沿父边走回去。
        }

        dfs1(v, u);
        // 先把儿子子树的三个状态算出来。

        f[2][u] = std::min(std::max(f[2][u], f[2][v] + 1), std::max(f[1][u], f[1][v]));
        // 第一项：u-v 不修铁路，v 子树内部可以随使用到 2 条，
        // 但多经过一条公路，所以 v 这一边整体 +1。
        // 第二项：u-v 修铁路，那么 u 之前最多只能已经用 1 条，v 也要给父边留一个端点。
        // 两种方案取更优的那个。

        f[1][u] = std::min(std::max(f[1][u], f[2][v] + 1), std::max(f[0][u], f[1][v]));
        // 同理，只不过目标变成恰好使用 1 条铁路边。

        f[0][u] = std::max(f[0][u], f[2][v] + 1);
        // 一条铁路都不能从 u 连向儿子，所以 u-v 只能当公路，直接 +1。
    }

    f[1][u] = std::min(f[0][u], f[1][u]);
    // 把“恰好 1 条”补成“不超过 1 条”。

    f[2][u] = std::min(f[1][u], f[2][u]);
    // 再把“不超过 2 条”补出来。
}

```

```

void dfs2(int u, int fa) {
    for (int i = 1; i <= f[2][1]; i++) {
        g[0][i][u] = 1;
        // 还没合并任何儿子时，什么边都不选只有这一种空方案。
    }

    for (int i = e.fst[u], v; ~i; i = e.nxt[i]) {
        v = e.to[i];

        if (v == fa) {
            continue;
        }

        dfs2(v, u);
        // 先统计完儿子子树。

        for (int i = 1; i <= f[2][1]; i++) {
            g[2][i][u] = add(mul(g[2][i][u], g[2][i - 1][v]), mul(g[1][i][u], g[1]
[i][v]));
            // 第一项: u-v 还是公路, v 这边允许的便利值要少 1。
            // 第二项: u-v 修铁路, u 和 v 两边都要各留一个铁路端点。

            g[1][i][u] = add(mul(g[1][i][u], g[2][i - 1][v]), mul(g[0][i][u], g[1]
[i][v]));
            // 和上面完全一样，只是总共只允许 1 个铁路端点。

            g[0][i][u] = mul(g[0][i][u], g[2][i - 1][v]);
            // 一个铁路端点都不能用, u-v 只能是公路。
            // 必须按 2、1、0 的顺序更新，不然会吃到这轮已经改过的状态。
        }
    }

    for (int i = 1; i <= f[2][1]; i++) {
        g[1][i][u] = add(g[0][i][u], g[1][i][u]);
        // 恰好 0 条 + 恰好 1 条 = 不超过 1 条。

        g[2][i][u] = add(g[1][i][u], g[2][i][u]);
        // 不超过 1 条 + 恰好 2 条 = 不超过 2 条。
    }
}

int main() {
    n = read();
    m = read();
    p = read();

    if (m != n - 1) {
        printf("-1\n-1\n");
        return 0;
        // 题目保证这张图本来就是森林。
        // 森林想全部连通成树，边数必须正好是 n-1。
    }
}

```

```

    for (int i = 1; i <= m; i++) {
        e.link(read(), read());
        // 建树。
    }

    dfs1(1, 0);
    // 第一遍求最小不便利值。

    dfs2(1, 0);
    // 第二遍在这个最优限制下统计方案数。

    printf("%d\n%d\n", f[2][1] - 1, g[2][f[2][1]][1]);
    // f 整体比真实值多 1，所以第一问要减回来。
    // 第二问直接取根节点、不超过两个铁路端点、最优不便利值限制下的方案数。

    return 0;
}

```

P3207 [HNOI2010] 物品调度

原题目链接： [P3207 物品调度](#)

题解链接： [洛谷专栏 smp5da0p](#)

代码解释

```

#include <bits/stdc++.h>
#define int long long
using namespace std;

int fa[100010], fac[100010], c[100010], pos[100010], vis[100010];
// fa: 环内并查集，跳过已经被占掉的位置。
// fac: 环间并查集，跳过已经彻底满掉的环。
// c: 题目生成出来的 c_i。
// pos[i]: 盒子 i 最后要去的位置。
// vis: 最后拆置换环时用。

int n, s, q, p, m, d;

int root(int x) { // 环内并查集，维护x位置是否被占
    if (fa[x] == x) return x;
    // 自己就是代表点，说明这里目前还是并查集的头。

    fa[x] = root(fa[x]);
    // 路径压缩，后面找空位时能直接跨过一整段已占位置。

    return fa[x];
}

```

```

int rootc(int x){//环间并查集，维护x所在环是否已满
    if(fac[x] == x)return x;

    fac[x] = rootc(fac[x]);
    // 这里压缩的是“哪些起点对应的 d 环已经满了”。

    return fac[x];
}

void init(){
    for(int i = 0; i <= n; i++)fa[i] = fac[i] = c[i] = pos[i] = vis[i] = 0;
    // 清掉上一组测试留下的状态。
}

void solve(){
    scanf("%lld%lld%lld%lld%lld",&n,&s,&q,&p,&m,&d);

    for(int i = 0; i < n; i++){
        if(i == 0)c[i] = 0;
        // 递推序列的真正初值就是 0。

        else c[i] = (c[i - 1] * q + p) % m;
        // 按题目公式生成 c_i。

        pos[i] = -1;
        // -1 表示这个盒子的最终位置还没求出来。
    }

    c[0] = s % n;//反正是第一个考虑，都是空位，直接把它记为s，它肯定会占用s位置
    // 把 0 位置看成一个“虚盒子”，它最后要去 s。
    // 这样真实盒子 + 虚盒子就会组成一个完整置换。

    //但是不能在其他i之前赋值，因为会影响后面的c[i]

    for(int i = 0; i <= n + 5; i++)fa[i] = fac[i] = i;
    // 两个并查集正式初始化，每个点先指向自己。

    for(int i = 0; i < n; i++){
        //j = (c[i] + y) % n,k = (c[i] + yi + xi * d) % n

        int j = c[i] % n;//m不一定小于n，即c[i]不一定小于n，记得取模
        // 先取 y_i=0 时所在的 d 环起点。

        while(rootc(j) != j || root(j) == n){//j所在环已经跑完，直接跳过
            // rootc(j)!=j: 这个起点之前已经被标记为没必要再看。
            // root(j)==n: 它所在的整条 d 环已经满了。

            if(rootc(j) == j)fac[j] = rootc((j + 1) % n);
            // 环已经满了，但这个 j 可能还没在 fac 里补标记，
            // 这里顺手把它接到下一个值得检查的位置。

            //这里连边是因为如果j所在的环已经满了，
            //但是最后一个被占的点为环上另一点k，
            //那么k在环间并查集标记了而j没有标记，

```



```
//这里需要补标记

j = rootc(j);
// 一次跳过已经无用的一整段 y。
}

int k = j;//x_min=0
// y 定下来以后, 先从最小的 x_i=0 开始找。

while(root(k) != k)k = root(k);//找到j所在环的空位置
// 用环内并查集跨过已经占掉的位置, 找到这个环里第一个空位。

pos[i] = k;
// 这个位置就是字典序要求下当前盒子的最终位置。

if(k != root((k + d) % n)){
    fa[k] = root((k + d) % n);
    // 占掉 k, 让以后再搜到 k 时直接跳到沿 d 走的下一个位置。
}else{
    fac[j] = rootc((j + 1) % n),fa[k] = n;
    // 再跳一步已经回到同一连续段, 说明这一整条 d 环刚好被填满。
    // fac 里把这个环标成以后不用再看, fa[k]=n 作为“环已满”的特殊标记。

    //再跳一步就从环上连续段的尾调到头了, 说明这个环满了
}
}

//注意概念上置换形成的环与处理pos时每次跳d单位的环不同

int ans = 0;

for(int i = 1; i < n; i++){
    if(vis[i] || pos[i] == i)continue;//i所在环已经处理或者自环情况就不用再处理了
    // 自环本来就在正确位置, 一次都不用动。

    int now = i,fl = 0,cnt = 0;
    // now 沿当前置换环走。
    // fl 记录这个环里有没有最终空位 s。
    // cnt 统计环长。

    while(!vis[now]){
        vis[now] = 1,cnt++;
        // 当前点算进这个置换环。

        if(now == s)fl = 1;
        // 包含 s 就等价于包含那个虚盒子所在的环。

        now = pos[now];
        // 沿“初始位置 -> 最终位置”继续走。
    }

    if(fl)ans += cnt - 1;
    // 空位本来就在这个环里, 沿环搬一圈只要 cnt-1 次。
```

```

        else ans += cnt + 1;
        // 空位不在环里，要先把空位借进来，最后再还出去，多两次。
    }

    printf("%lld\n",ans);
}

signed main(){
    int T;
    scanf("%lld",&T);

    while(T--){
        init(),solve();
        // 这里先清空状态，再处理这一组数据。
    }

    return 0;
}

```

P3149 排序

原题目链接： [P3149 排序](#)

题解链接： [洛谷专栏 xivufe6j](#)

代码解释

```

#include <bits/stdc++.h>
using namespace std;

long long n,m,sum[400005],x,b[400005],ans,xx,w,c[400005];
// sum[v]: 一开始先存离散化等级恰好为 v 的元素贡献了多少逆序对，后面会做前缀和。
// b: 树状数组。
// c[i]: 原位置 i 的离散化等级。
// xx: 到目前为止，操作过的最大身高阈值对应的原位置。
// ans、w 在这份代码里没有参与答案计算，保留声明以维持代码结构。

struct node{
    long long id,cnt;
    // id 是原位置，cnt 是身高。

    bool operator<(const node i)const{return cnt<i.cnt;}
    // sort 时直接按身高从小到大排。
}a[400005];

bool operator<(const long long i,const node j){return i<j.cnt;}

int add(long long i,long long j)
{

```

```
while(i<=n)
{
    b[i]+=j;
    // 把当前位置的计数加到所有包含它的树状数组节点里。

    i+=i&-i;
    // 往父节点跳。
}

return 0;
}

long long su(long long i)
{
    long long ans=0;

    while(i>0)
    {
        ans+=b[i];
        // 累加前缀计数。

        i-=i&-i;
        // 往上一个负责更小前缀的节点跳。
    }

    return ans;
}

int main(){
    cin>>n>>m;

    for(long long i=1;i<=n;i++)
        cin>>a[i].cnt,a[i].id=i;
    // 保存身高和原下标，排序以后还要写回原位置。

    std::sort(a+1,a+n+1);
    // 先按身高排序做离散化。

    c[a[1].id]=1;
    // 最小身高的等级从 1 开始。

    for(long long i=2;i<=n;i++)
        c[a[i].id]=(a[i].cnt!=a[i-1].cnt)+c[a[i-1].id]; //离散化
    // 身高变大就把等级 +1，相同身高继续沿用前一个等级。

    for(long long i=n;i>=1;i--)
    {
        sum[c[i]]+=su(c[i]-1);
        // 从右往左扫。
        // 树状数组里已经是 i 右边的人，su(c[i]-1) 就是右边严格比当前人矮的数量。
        // 这些逆序对都记到当前身高等级的贡献里。

        add(c[i],1);
        // 把当前位置加入树状数组，给更左边的人统计。
    }
}
```

```

    }

    n=c[a[n].id];
    // 后面只需要枚举离散化后的最大等级，不再需要原人数。

    for(long long i=2;i<=n;i++)
        sum[i]+=sum[i-1];
        // 做前缀和后，sum[v] 变成“所有身高等级 <=v 的人一共贡献多少逆序对”。

    cout<<sum[n]<<endl;
    // 任何操作之前，sum[n] 就是整段序列的逆序对总数。

    c[0]=-1e10;
    // xx 全局初值是 0。
    // 给 c[0] 一个极小值，保证第一次询问正常更新 xx。

    for(long long i=1;i<=m;i++)
    {
        cin>>x;

        if(c[x]>c[xx]) xx=x;//如果c[x]<c[xx]就不会改变答案
        // 已经排过某个更大的身高阈值后，再排一个更小阈值不会继续影响答案。
        // 所以只需要一直记住历史上最大的离散化等级。

        cout<<sum[n]-sum[c[xx]]<<endl;
        // 被当前最大阈值覆盖到的那些身高贡献已经全部消掉。
        // 总逆序对减去这部分，就是现在还剩的逆序对。
    }

    return 0;
}

```

P1380 T 型骨牌

原题目链接： [P1380 T 型骨牌](#)

题解链接： [洛谷专栏 2urnblrk](#)

代码解释

```

#include<bits/stdc++.h>
using namespace std;

int n,m,a[15][15],d[15][15],ans;
// a[x][y]: 当前搜索分支里这个格子有没有被骨牌占掉。
// d[x][y]: 以前搜到中心 (x,y) 时达到过的较好放置数量，用来剪枝。
// ans: 目前找到的最大骨牌数。

void dfs(int x,int y,int s)

```

```
{
    if(x==n)//边界条件
    {
        ans=max(ans,s);
        // 中心最多只会枚举到 n-1 行。
        // x==n 说明所有候选中心都处理完，用当前数量更新答案。

        return;
    }

    if(!a[x-1][y-1]&&!a[x-1][y]&&!a[x-1][y+1]&&!a[x+1][y]&&!a[x][y])
    {
        // 第一种方向：
        // ***
        //  *
        //  *
        // 五个位置都没被占才能放。

        a[x-1][y-1]=a[x-1][y]=a[x-1][y+1]=a[x+1][y]=a[x][y]=1;
        // 先把这一块真正占到棋盘上。

        if(s+1>=d[x][y])
        {
            // 放完这一块以后没有明显劣于以前到达这里的状态，才继续搜。

            d[x][y]=max(d[x][y],s);
            // 更新这个中心位置记录过的较好状态。

            if(y==m-1) dfs(x+1,2,s+1);
            // 已经是这一行最后一个合法中心，换到下一行第 2 列。

            else dfs(x,y+1,s+1);
            // 否则继续看右边的中心。
        }

        a[x-1][y-1]=a[x-1][y]=a[x-1][y+1]=a[x+1][y]=a[x][y]=0;
        // 回溯，撤掉刚才这块骨牌。
    }

    if(!a[x+1][y-1]&&!a[x+1][y]&&!a[x+1][y+1]&&!a[x-1][y]&&!a[x][y])
    {
        // 第二种方向：
        //  *
        //  *
        // ***

        a[x+1][y-1]=a[x+1][y]=a[x+1][y+1]=a[x-1][y]=a[x][y]=1;
        // 标记这五个格子已经被占。

        if(s+1>=d[x][y])
        {
            d[x][y]=max(d[x][y],s);

            if(y==m-1) dfs(x+1,2,s+1);
```

```
        // 行尾就换行。

        else dfs(x,y+1,s+1);
        // 否则继续往右。
    }

    a[x+1][y-1]=a[x+1][y]=a[x+1][y+1]=a[x-1][y]=a[x][y]=0;
    // 回溯恢复。
}

if(!a[x-1][y-1]&&!a[x][y-1]&&!a[x][y+1]&&!a[x+1][y-1]&&!a[x][y])
{
    // 第三种方向:
    // *
    // ***
    // *

    a[x-1][y-1]=a[x][y-1]=a[x][y+1]=a[x+1][y-1]=a[x][y]=1;
    // 占掉左边竖三格、中心和右边一格。

    if(s+1>=d[x][y])
    {
        d[x][y]=max(d[x][y],s);

        if(y==m-1) dfs(x+1,2,s+1);

        else dfs(x,y+1,s+1);
    }

    a[x-1][y-1]=a[x][y-1]=a[x][y+1]=a[x+1][y-1]=a[x][y]=0;
    // 撤销这一块, 继续试别的方向。
}

if(!a[x-1][y+1]&&!a[x][y-1]&&!a[x][y+1]&&!a[x+1][y+1]&&!a[x][y])
{
    // 第四种方向:
    // *
    // ***
    // *

    a[x-1][y+1]=a[x][y-1]=a[x][y+1]=a[x+1][y+1]=a[x][y]=1;
    // 占掉右边竖三格、中心和左边一格。

    if(s+1>=d[x][y])
    {
        if(y==m-1) dfs(x+1,2,s+1);

        else dfs(x,y+1,s+1);

        d[x][y]=max(d[x][y],s);
        // 这一种方向里, 这里把 d 的更新写在递归之后。
        // 保持代码原有顺序。
    }
}
```

```

        a[x-1][y+1]=a[x][y-1]=a[x][y+1]=a[x+1][y+1]=a[x][y]=0;
        // 回溯。
    } // 四种骨牌

    if(y==m-1) dfs(x+1,2,s); // 换行
    // 前四个分支都是“这里放一块”。
    // 最后还必须保留“这个中心什么都不放”的分支，否则会漏解。

    else dfs(x,y+1,s);
    // 不放的话 s 不变，继续枚举下一个中心。
}

int main()
{
    cin>>n>>m;

    if(n<3 || m<3) // 特判
    {
        cout<<0;
        // 棋盘连最基本的 T 形跨度都放不下，直接 0。

        return 0;
    }

    dfs(2,2,0);
    // 最外面一圈不可能当中心，所以从 (2,2) 开始。
    // 一开始一块都没放。

    cout<<ans;

    return 0;
}

```

P6141 [JSOI2013] 贪心的导游

原题目链接： [P6141 贪心的导游](#)

题解链接： [_hud 《P6141 贪心的导游》](#)

代码解释

```

#include <bits/stdc++.h>
using namespace std;

const int N = 1e6 + 10, M = 5e4 + 10, V = 1e3 + 10;

// cnt[x] 记当前莫队区间里 x 出现了多少次
// len 是莫队块长，ans 按原询问编号存答案
int n, m, a[N], cnt[V], len, ans[M];

```

```

// 这里只有“这个数在不在当前区间”有用，出现十次和出现一次对答案没区别
bitset<V> f;

struct Q {
    int l, r, p, id;

    bool operator< (const Q &x) const {
        // 看当前询问左端点落在哪一块
        int lid = l / len;

        // 不同块先按左端点排，等价于按块编号排
        if(lid ^ x.l / len) return l < x.l;

        // 同一块里让 r 一块正着、一块反着跑
        // 经典奇偶块优化，不然右指针会反复横跳
        else return lid & 1 ? r > x.r : r < x.r;
    }
} q[M];

inline void add(int x) {
    // cnt[x] 原来是 0，说明 x 是第一次进入当前区间
    // 这时候才需要把存在标记打开
    if(!cnt[x]++) f[x] = 1;
}

inline void del(int x) {
    // 先减出现次数；减完变成 0，区间里就真的没有 x 了
    if(--cnt[x]) f[x] = 0;
}

signed main() {
    cin.tie(0), cout.tie(0) -> sync_with_stdio(0);

    cin >> n >> m;

    // 这里取块长
    // 大意是让左右指针移动次数和询问数比较平衡
    len = n / sqrt(m);

    for(int i = 1; i <= n; ++i) cin >> a[i];

    for(int i = 1; i <= m; ++i) {
        // id 留下来，因为后面 sort 以后询问顺序就乱了
        cin >> q[i].l >> q[i].r >> q[i].p, q[i].id = i;

        // 题目里的商店编号是 0 开始，这份代码内部按 1 开始存
        ++q[i].l, ++q[i].r;

        // 这题有个挺阴的地方：数据里真会出现 u > v
        if(q[i].l > q[i].r) swap(q[i].l, q[i].r);
    }

    // 按莫队顺序重排询问

```



```

    sort(q + 1, q + m + 1);

    // pl=1,pr=0 表示一开始维护的是空区间
    for(int i = 1, pl = 1, pr = 0; i <= m; ++i) {
        // 四个 while 就是在把 [pl,pr] 硬挪到当前询问的 [l,r]
        while(pl > q[i].l) add(a[--pl]);
        while(pr < q[i].r) add(a[++pr]);
        while(pl < q[i].l) del(a[pl++]);
        while(pr > q[i].r) del(a[pr--]);

        int res = 0;

        // 值域才 1000, 别想复杂了, 直接整个扫掉
        for(int j = 1; j < V; ++j)
            if(f[j]) {
                // 当前区间存在 j, 就拿 j % p 更新答案
                res = max(res, j % q[i].p);

                // 模 p 的余数最大只能是 p-1
                // 已经摸到上界就可以直接收工了
                if(res == q[i].p - 1) break;
            }

        // 当前是莫队排序后的第 i 个询问, 要存回它原来的位置
        ans[q[i].id] = res;
    }

    // 最后按输入顺序输出
    for(int i = 1; i <= m; ++i) cout << ans[i] << '\n';

    return 0;
}

```

P6813 「MCOI-02」 Glass 玻璃

原题目链接: [P6813 Glass 玻璃](#)

题解链接: [洛谷题解](#)

代码解释

```

#include<bits/stdc++.h>
#define ll long long
using namespace std;
const int MAXN = 2e6 + 7;
const int MAXM = 4e6 + 7;
const int P = 998244353;
// dp[u] 是子树内所有点到 u 的点权和; f[u] 是子树外所有点到 u 的点权和。
// siz[u] 记录子树大小, ans 按边统计两侧路径的贡献。

```

```

int n, home[MAXN], nxt[MAXM], to[MAXM], w[MAXM], ccnt, a[MAXN], siz[MAXN];
ll dp[MAXN], f[MAXN], ans;

namespace fastIO {
#define BUF_SIZE (1<<21)
    static char buf[BUF_SIZE], *p1 = buf + BUF_SIZE, *pend = BUF_SIZE + buf;
    inline char nc() {
        if(p1 == pend) {
            p1 = buf;
            pend = buf + fread(buf, 1, BUF_SIZE, stdin);
        }
        return *p1++;
    }
    inline int read() {
        char s = nc();
        int x = 0;
        for(; !isdigit(s); s = nc());
        for(; isdigit(s); s = nc()) x = (x << 1) + (x << 3) + s - '0';
        return x;
    }
}
using namespace fastIO;

inline void ct(int x, int y, int z) {
    // 链式前向星存边；每条无向边会调用两次。
    ccnt++;
    nxt[ccnt] = home[x];
    home[x] = ccnt;
    to[ccnt] = y;
    w[ccnt] = z;
}

inline void add(ll &x, ll y) {
    x += y;
    if(x >= P) x -= P;
}

inline void dfs1(int u, int F) {
    siz[u] = 1;
    for(int i = home[u]; i; i = nxt[i]) {
        int v = to[i];
        if(v == F) continue;
        dfs1(v, u);
        add(dp[u], dp[v]);
        siz[u] += siz[v];
    }
    // a[u] 会被子树中的每个点经过一次，所以贡献是 siz[u]*a[u]。
    add(dp[u], 1ll * siz[u] * a[u] % P);
}

inline void dfs2(int u, int F) {
    for(int i = home[u]; i; i = nxt[i]) {
        int v = to[i];
        if(v == F) continue;
    }
}

```

```

// 去掉 v 子树后, 先得到 u 的其余子树贡献, 再补上 u 及 u 的子树外部分。
f[v] = ((dp[u] - dp[v] - 111 * siz[v] * a[u] % P + P) % P
        + f[u] + 111 * (n - siz[u]) * a[u] % P) % P;
add(f[v], P);
// dp[v] 与 f[v] 分别是边两侧到端点的点权和, 乘上另一侧大小后就是所有路径贡献。
add(ans, ((111 * f[v] * siz[v] % P * w[i] % P
            + 111 * w[i] * dp[v] % P * (n - siz[v]) % P) % P + P) % P);
dfs2(v, u);
}
}

int main() {
    n = read();
    for(int i = 1; i <= n; ++i) a[i] = read();
    for(int i = 1, x, y, z; i < n; ++i) {
        x = read();
        y = read();
        z = read();
        ct(x, y, z);
        ct(y, x, z);
    }
    dfs1(1, 0);
    dfs2(1, 0);
    printf("%d\n", 111 * ans * 2 % P);
    return 0;
}

```

P6830 [IOI 2020] Connecting Supertrees

原题目链接: [P6830 Connecting Supertrees](#)

题解链接: [s_r_f 《P6830 Connecting Supertrees》](#)

代码解释

```

#include <bits/stdc++.h>
#include "supertrees.h"
using namespace std;

const int N = 1001;

// cur[x] 是 x 属于哪个大的连通块
// G 就是题目给的 p 矩阵, ans 是最后构造出来的邻接矩阵
int n, cur[N], cntc, G[N][N], ans[N][N];

// p 临时存当前连通块里的原点编号
// fa 是后面给 p=1 分组用的并查集
int p[N], l, fa[N];

```

```

inline int Find(int x){
    // 普通并查集带路径压缩
    return x == fa[x] ? x : (fa[x] = Find(fa[x]));
}

inline void Merge(int x,int y){
    // p=1 的两个点必须在同一棵树里，直接并起来
    fa[Find(x)] = Find(y);
}

inline void add_edge(int x,int y){
    // 真正往答案图里塞一条无向边
    ans[x][y] = ans[y][x] = 1;
}

inline bool solve(int c){
    int i,j;
    bool is = 0;

    // 把第 c 个大连通块里的所有点捞出来
    // 后面为了方便，局部重新编号成 1...l
    for (l = 0,i = 1; i <= n; ++i)
        if (cur[i] == c) p[++l] = i;

    // 如果整个块里面全是 1，就只需要造一棵树
    // 只要看见一个不是 1 的关系，就说明这个块需要那个唯一的环
    for (i = 1; i <= l; ++i)
        for (j = 1; j <= l; ++j)
            if (G[p[i]][p[j]] != 1) is = 1;

    // 单独一个点，啥边都不用连
    if (l == 1) return 1;

    if (l == 2){
        // 两个点之间要两条路是不可能的，简单图又不能连重边
        if (G[p[1]][p[2]] > 1) return 0;

        // 要一条路就直接牵根线
        add_edge(p[1],p[2]);
        return 1;
    }

    if (!is){
        // 整个连通块任意两点都要求恰好一条路径
        // 直接把代表点连成链，构造一棵满足连通性的树。
        for (i = 1; i < l; ++i) add_edge(p[i],p[i+1]);
        return 1;
    }

    // 有 p=2 的情况，开始处理“树块 + 一个环”
    for (i = 1; i <= l; ++i) fa[i] = i;

    // 所有要求只有一条路径的点必须被绑在同一块里
    for (i = 1; i <= l; ++i)

```

```

        for (j = 1; j <= l; ++j)
            if (G[p[i]][p[j]] == 1) Merge(i,j);

// 把根压干净, 下面直接比较 fa[i] 会方便很多
for (i = 1; i <= l; ++i) fa[i] = Find(i);

for (i = 1; i <= l; ++i)
    for (j = 1; j <= l; ++j)
        // 已经被 p=1 合进同一块, 题目却又说这俩之间要两条路径, 矛盾
        if (fa[i] == fa[j] && G[p[i]][p[j]] == 2) return 0;

for (i = 1; i <= l; ++i)
    for (j = 1; j <= l; ++j)
        // 反过来也一样: 明明被分到了不同树块, 却要求只有一条路径
        if (fa[i] != fa[j] && G[p[i]][p[j]] == 1) return 0;

// x 用来在一个 p=1 块内部拉链
// lst 是上一个树块的代表, fir 记环上的第一个代表
// cnt 统计最后到底有多少个树块上环
int x = 0, lst = 0, fir, cnt = 0;

for (i = 1; i <= l; ++i){
    x = is = 0;

    // i 不一定是某个并查集的根
    // 先看看有没有点的根真的是 i
    for (j = 1; j <= l; ++j)
        if (fa[j] == i) is = 1;

    if (!is) continue;

    ++cnt;

    // 同一个 p=1 块里面随便接成一棵树即可
    // 这里同样用链连接各个代表点, 构造最简单的环外结构。
    for (j = 1; j <= l; ++j)
        if (j != i && fa[j] == i){
            if (x) add_edge(p[x], p[j]);
            x = j;
        }

    // 把链最后再挂回这个并查集根对应的原点
    if (x) add_edge(p[x], p[i]);

    // 每个 p=1 块拿根 p[i] 当代表
    // 先把所有代表依次串起来
    if (lst) add_edge(p[lst], p[i]);
    else fir = i;

    lst = i;
}

// 一个简单环至少三个点
// 两个树块想制造两条路径只能靠重边, 但题目不允许

```

```

    if (cnt < 3) return 0;

    // 最后首尾一接，环闭上了
    add_edge(p[lst],p[fir]);

    return 1;
}

inline void getcur(int x,int c){
    // 按 G[x][i] 是否非 0 来找题目要求中的连通块
    cur[x] = c;

    for (int i = 1; i <= n; ++i)
        if (x != i && G[x][i] && !cur[i])
            getcur(i,c);
}

inline int MAIN(){
    int i,j;

    // 先把所有 p>0 连起来的大连通块找出来
    for (i = 1; i <= n; ++i)
        if (!cur[i]) getcur(i,++cntc);

    // 分属两个不同大连通块，却要求有路径，肯定不合法
    for (i = 1; i <= n; ++i)
        for (j = 1; j <= n; ++j)
            if (cur[i] != cur[j] && G[i][j]) return 0;

    // 已经在同一个大连通块里，却要求路径数为 0，也矛盾
    for (i = 1; i <= n; ++i)
        for (j = 1; j <= n; ++j)
            if (cur[i] == cur[j] && !G[i][j]) return 0;

    // 每个连通块独立构造
    for (i = 1; i <= cntc; ++i)
        if (!solve(i)) return 0;

    vector<vector<int>> >Ans;
    vector<int>ret;
    Ans.clear();

    for (i = 1; i <= n; ++i){
        ret.resize(n);

        // ans 内部用的是 1-index，这里转回题目接口要的 0-index vector
        for (j = 0; j < n; ++j)
            ret[j] = ans[i][j+1];

        // 这里不需要保留调试输出
        // 正式构造真正需要的只是 Ans
        Ans.push_back(ret);
    }
}

```

```

// 把构造好的图交给交互库
build(Ans);

return 1;
}

int construct(vector<vector<int> > p){
    int i,j;

    // 先拿到点数
    n = p[0].size();

    // 前面已经分析过：这种结构不可能恰好制造 3 条简单路径
    // 看见 3 直接判死
    for (i = 0; i < n; ++i)
        for (j = 0; j < n; ++j)
            if (p[i][j] == 3) return 0;

    // 代码主体使用 1-index，因此这里复制矩阵
    for (i = 1; i <= n; ++i)
        for (j = 1; j <= n; ++j)
            G[i][j] = p[i-1][j-1];

    return MAIN();
}

```

P6858 深海少女与胖头鱼

原题目链接： [P6858 深海少女与胖头鱼](#)

题解链接： [Fairicle 《P6858 深海少女与胖头鱼》](#)

代码解释

```

#include "bits/stdc++.h"
using namespace std;

#define ri register int
#define ll long long
#define N 1000010
#define mod 998244353

// n 是一开始带盾鱼数量，m 是没盾鱼数量
// inv2 后面拿来算公式里的 /2
ll m,n,inv2;

inline ll ksm(ll x,int y){
    ll res=1;

```

```

// 先压到模数里，尤其 n 最大能到 1e14
x%=mod;

while(y){
    // 快速幂正常做
    // 这里主要是用费马小定理求各种分母的逆元
    if(y&1) res*=x,res%=mod;

    x*=x,x%=mod;
    y>>=1;
}

return res;
}

inline ll f(ll x){
    // 前面已经得到:
    //  $f(x) = (x^2 + 3x - 2) / 2$ 
    return (x*x+3ll*x-2ll+mod)%mod*inv2%mod;
}

int main(){
    cin>>n>>m;

    // 后面所有关于 n 的计算都只在模意义下进行
    // 所以先把这个 1e14 级别的数压下来
    n%=mod;

    // 2 的逆元
    inv2=ksm(2,mod-2);

    // bas 维护“前面连续打中的全是不带盾鱼”的概率
    // ans 累答案
    // cnt 在后面的实际计算里没有使用，是冗余变量
    ll bas=1,ans=0,cnt=1;

    // 第一发就直接打中带盾鱼:
    // 概率是  $n/(n+m)$ 
    // 打完这一发以后进入  $f(n+m)$  那类状态
    // 外面的 +1 是当前这一发攻击本身
    ll del=n*ksm(m+n,mod-2)%mod*(f(m+n)+1)%mod;

    ans=del;

    for(ri i=1;i<=m;++i){
        // 这个变量后面没有参与答案，保留以对应代码结构
        ++cnt;

        // 前 i 发都先打在没盾鱼身上的概率
        //
        // 每打死一条普通鱼:
        // 分子里普通鱼数量从  $m, m-1, \dots$  往下掉
        // 分母里总活鱼数量也一起往下掉
        bas=bas*(m-i+1)%mod*ksm(m-i+1+n,mod-2)%mod;
    }
}

```



```

        // 前 i 次全打普通鱼以后，第 i+1 次终于打中带盾鱼
        //
        // bas: 前面全打普通鱼
        // n/(m+n-i): 这一发打中带盾鱼
        // f(m+n-i): 进入特殊局面后的剩余期望
        // i+1: 到这里已经实际攻击了 i+1 次
        ll del=bas*n%mod*ksm(m+n-i,mod-2)%mod
            *(f(m+n-i)+i+1)%mod;

        // 把“第一次碰到带盾鱼是在第 i+1 发”这种情况加进去
        ans=(ans+del)%mod;
    }

    cout<<ans;

    return 0;
}

```

P6059 [加油武汉] 居家隔离

原题目链接: [P6059 居家隔离](#)

题解链接: [洛谷题解](#)

代码解释

```

#include<bits/stdc++.h>
#define del(a,i) memset(a,i,sizeof(a))
#define ll long long
#define inl inline
#define il inl void
#define it inl int
#define ill inl ll
#define re register
#define ri re int
#define rl re ll
#define INF 0x3f3f3f3f
#define lowbit(x) (x&(-x))
#define mid ((l+r)>>1)
using namespace std;
template<class T>il read(T &x){
    x=0;int f=1;char k=getchar();
    for(;k>'9' || k<'0';k=getchar()) if(k=='-') f=-1;
    for(;k>='0'&&k<='9';k=getchar()) x=x*10+k-'0';
    x*=f;
}
template<class T>il _print(T x){
    if(x>9) _print(x/10);

```

```

    putchar(x%10+'0');
}
template<class T>il print(T x){
    if(x<0) putchar('-'),x=-x;
    _print(x);
}
it qpow(int x,int k,int mod){
    int res=1,bas=x;
    while(k){
        if(k&1) res=1ll*res*bas%mod;
        bas=1ll*bas*bas%mod,k>>=1;
    }
    return res;
}
const int N = 1e3+5,mod = 998244353;
// val 排序后, sum[i] 是前 i 个数的前缀和。
// fac、ifac 用来在枚举前 m 个位置的最大值时快速算组合数。
int n,fac[N],ifac[N],val[N],sum[N];
it add(int x,int y){return x+y>=mod?x+y-mod:x+y;}
it mul(int x,int y){return 1ll*x*y%mod;}
il inc(int &x,int y){x=add(x,y);}
it C(int n,int m){return m>n?0:mul(mul(ifac[m],ifac[n-m]),fac[n]);}
il Solve(int m){
    int res=0,div=fac[n];
    // i<n: 前 m 个数的最大值是第 i 小, 答案取自 i 之后最先出现的数。
    for(ri i=m;i<n;++i){
        int tval=add(sum[n],mod-sum[i]);
        // 后缀和代替枚举最后取到的数, 后面三项是剩余位置的排列数。
        tval=mul(tval,C(i-1,m-1));
        tval=mul(tval,C(n-m,i-m));
        tval=mul(tval,fac[m]);
        tval=mul(tval,fac[i-m]);
        tval=mul(tval,fac[n-i-1]);
        inc(res,tval);
    }
    // i=n 时已经没有更大的数, 最后留下的数单独计算。
    int tval=mul(C(n-2,m-1),fac[m]);
    tval=mul(tval,sum[n-1]);
    tval=mul(tval,fac[n-m-1]);
    inc(res,tval);
    print(mul(res,qpow(div,mod-2,mod)),putchar(' '));
}
int main(){
    read(n),fac[0]=1;
    // 先预处理阶乘和逆阶乘, 所有询问共用这部分结果。
    for(ri i=1;i<=n;++i) fac[i]=mul(fac[i-1],i);
    ifac[n]=qpow(fac[n],mod-2,mod);
    for(ri i=n-1;i>=0;--i) ifac[i]=mul(ifac[i+1],i+1);
    for(ri i=1;i<=n;++i) read(val[i]);
    sort(val+1,val+1+n);
    for(ri i=1;i<=n;++i) sum[i]=add(sum[i-1],val[i]);
    for(ri i=1;i<n;++i) Solve(i);
    return 0;
}

```

```
}
```

P4949 最短距离

原题目链接: <https://www.luogu.com.cn/problem/P4949>

题解链接: [hyfhaha 《题解 P4949 【最短距离】》](#)

代码解释

```
#include<bits/stdc++.h>
#define maxn 4000001
#define L(x) (x<<1)
#define R(x) ((x<<1)|1)
using namespace std;

int tree[maxn],tag[maxn];

// rev: dfs 序反查原点
// dep: 深度
// size: 子树大小
// seg: 每个点在线段树里的 dfs 序
// top: 当前重链链顶
// son: 重儿子
// father: 父亲
int rev[maxn],dep[maxn],size[maxn],seg[maxn],top[maxn],son[maxn],father[maxn];

int n,m,root,x,y,z,a[maxn],visx[maxn],visy[maxn],tot,mode;

// 这里是链式前向星。
// 因为每条树边会正反各存一次，所以后面“边编号乘 2”就是从这里来的。
int cnt,from[maxn],to[maxn],Next[maxn],head[maxn];

// Gx,Gy: 那条额外边的两个端点
// Gz: 额外边当前的边权
// Gd: 额外边在输入中的编号
int Gx,Gy,Gz,Gd;

// fa 是并查集。
// x,y,z 则把输入的每条原边完整存下来，修改边权的时候要用。
int fa[maxn],X[maxn],Y[maxn],Z[maxn];

int find(int x){
    if(fa[x]==x)return x;

    // 顺手路径压缩，不然后面判环会慢。
    else return fa[x]=find(fa[x]);
}
```

```
bool merge(int x,int y){
    int i=find(x),j=find(y);

    // 两个点已经在同一个连通块里，
    // 这条边再接进去就会形成唯一的环，所以不能放进生成树。
    if(i==j)return false;

    // 不在一个连通块就正常合并，这条边会成为树边。
    fa[i]=j;
    return true;
}

void add(int x,int y){
    cnt++;

    // 记录这条有向边的起点和终点。
    from[cnt]=x;
    to[cnt]=y;

    // 插到 x 的邻接表最前面。
    Next[cnt]=head[x];
    head[x]=cnt;
}

void update(int node,int begin,int end,int x,int y,int val){
    // 当前线段和要修改的区间完全没交集。
    if(begin>y||end<x)return;

    if(begin>=x&&end<=y){
        // 实际上这里后面都是单点修改，
        // 所以走到完全覆盖时直接把值改掉。
        tree[node]=val;
        return;
    }else{
        int mid=(begin+end)>>1;

        if(x<=mid)update(L(node),begin,mid,x,y,val);
        if(y>mid)update(R(node),mid+1,end,x,y,val);

        // 子节点改完以后重新维护区间和。
        tree[node]=tree[L(node)]+tree[R(node)];
    }
}

int query(int node,int begin,int end,int x,int y){
    if(begin>=x&&end<=y){
        // 当前线段完全在询问范围里，直接拿整段。
        return tree[node];
    }else{
        int mid=(begin+end)>>1,sum=0;

        // 询问区间碰到左半边就向左找。
        if(x<=mid)sum+=query(L(node),begin,mid,x,y);
```

```
        // 碰到右半边就往右找。
        if(y>mid)sum+=query(R(node),mid+1,end,x,y);

        return sum;
    }
}

int dfs1(int x){
    // 子树里至少有自己一个点。
    size[x]=1;

    // 深度直接由父亲推下来。
    dep[x]=dep[father[x]]+1;

    for(int i=head[x];i!=-1;i=Next[i]){
        int v=to[i],big=0;

        // 无向边会反着走回来，父亲不能再搜。
        if(father[x]==v)continue;

        // 先把父子关系定下来。
        father[v]=x;

        // big 就是 v 这棵子树的大小。
        big=dfs1(v);

        size[x]+=big;

        // 谁的子树最大，谁就是 x 的重儿子。
        if(big>size[son[x]])son[x]=v;
    }

    return size[x];
}

void dfs2(int x){
    if(son[x]){
        // 重儿子紧跟父亲分配 dfs 序，
        // 这样一整条重链在线段树上就是连续的一段。
        seg[son[x]]=++seg[0];

        // 重儿子和父亲属于同一条重链。
        top[son[x]]=top[x];

        // 保存 dfs 序对应哪个原点。
        rev[seg[0]]=son[x];

        dfs2(son[x]);
    }

    for(int i=head[x];i!=-1;i=Next[i]){
        int v=to[i];

        // 已经有 top 的点就是父亲或者刚处理过的重儿子，
```

```

        // 剩下的才是轻儿子。
        if(!top[v]){
            seg[v]=++seg[0];

            // 轻边下面要另开一条链，所以链顶就是自己。
            top[v]=v;

            rev[seg[0]]=v;

            dfs2(v);
        }
    }
}

int linkquery(int x,int y){
    int fx=top[x],fy=top[y],ans=0;

    // 只要还不是一条重链上，就把深度更大的那条链往上跳。
    while(fx!=fy){
        if(dep[fx]<dep[fy]){
            swap(x,y);
            swap(fx,fy);
        }

        // fx 到 x 这一整段在 dfs 序上连续，可以一次线段树求和。
        ans+=query(1,1,seg[0],seg[fx],seg[x]);

        // 跳到这条链链顶的父亲。
        x=father[fx];
        fx=top[x];
    }

    // 到这里 x,y 已经在同一条重链上。
    // 让 x 始终是深度浅的那个。
    if(dep[x]>dep[y])swap(x,y);

    // 先把 x 到 y 对应的所有点权加进来。
    ans+=query(1,1,seg[0],seg[x],seg[y]);

    // 这里维护的是“边权下放到深度更大的端点”。
    // x 是这段路径的 LCA，它身上的点权其实是 father[x] -> x，
    // 不属于 x 到 y 的路径，因此最后必须减掉。
    ans-=query(1,1,seg[0],seg[x],seg[x]);

    return ans;
}

void change(int x,int y){ //修改边权
    // add(x,y),add(y,x) 后，
    // 输入第 x 条树边对应链式前向星里的第 2x-1、2x 两条有向边。
    // 这里直接取第 2x 条有向边。
    x*=2; //记得乘2

    // 边权被放在深度更大的端点上，

```

```

// 所以先找出这条边的儿子。
x=dep[from[x]]>dep[to[x]]?from[x]:to[x]; //因为边权钦定的是深度大的点

// 然后在线段树上改单点。
update(1,1,seg[0],seg[x],seg[x],y); //直接改
}

int main(){
// 链式前向星用 -1 表示一条邻接表到头。
memset(head,-1,sizeof(head));

scanf("%d%d",&n,&m);

// 随便拿 1 当树根。
root=1;

// 并查集初始化。
for(int i=1;i<=n;i++)fa[i]=i;

for(int i=1;i<=n;i++){
    scanf("%d%d%d",&x,&y,&z);

    if(merge(x,y)){
        // 不成环的边留下来，构成一棵生成树。
        add(x,y);
        add(y,x);
    }else{
        // n 个点 n 条边且原图连通，所以只会多出这么一条边。
        // 单独把它记下来，不塞进树里。
        Gx=x;
        Gy=y;
        Gz=z;
        Gd=i;

        // 虽然没真的 add，但仍然把 cnt 空出两个位置。
        // 这样第 i 条输入边依然和编号 2i 对得上，
        // change 里的 x*=2 才不会乱掉。
        cnt+=2;
    }

    // 修改操作按原始边编号给，
    // 所以每条输入边都得留一份。
    X[i]=x;
    Y[i]=y;
    Z[i]=z;
}

// 第一遍 dfs 求 father、dep、size、son。
dfs1(root);

// 根节点的 dfs 序单独先放进去。
seg[root]=++seg[0];
rev[seg[0]]=root;
top[root]=root;

```

```
// 第二遍 dfs 正式做重链划分。
dfs2(root);

// 代码没有单独 build,
// 直接把每条树边逐条 change 进去。
// 唯一那条额外边当然不能放在线段树里。
for(int i=1;i<=n;i++){
    if(Gd!=i)change(i,Z[i]);
}

for(int i=1;i<=m;i++){
    scanf("%d%d%d",&mode,&x,&y);

    if(mode==1){
        if(x==Gd){
            // 改到额外边, 直接改 Gz 即可,
            // 因为它根本不在树剖的线段树里。
            Gz=y;
        }else{
            // 普通树边就在对应点权上修改。
            change(x,y);
        }
    }else{
        int ans;

        // 第一种走法: 完全沿生成树走。
        ans=linkquery(x,y);

        // 第二种走法: 一定经过那条额外边。
        //
        // 额外边可以按 Gx -> Gy 走:
        // x -> Gx + GxGy + Gy -> y
        //
        // 也可以反过来 Gy -> Gx 走:
        // x -> Gy + GyGx + Gx -> y
        //
        // 三种路线取最短的就是原图最短路。
        ans=min(ans,Gz+min(
            linkquery(x,Gx)+linkquery(Gy,y),
            linkquery(x,Gy)+linkquery(Gx,y)
        ));

        printf("%d\n",ans);
    }
}
}
```

P6046 纯粹容器

原题目链接: <https://www.luogu.com.cn/problem/P6046>

题解链接: [ZCETHAN 《P6046 纯粹容器》](#)

代码解释

```
#include<bits/stdc++.h>
#define ll long long
#define inf (1<<30)
#define INF (1ll<<60)
#define pb emplace_back
#define pii pair<int,int>
#define mkp make_pair
#define fi first
#define se second
#define all(a) a.begin(),a.end()
#define siz(a) (int)a.size()
#define clr(a) memset(a,0,sizeof(a))
#define rep(i,j,k) for(int i=(j);i<=(k);i++)
#define per(i,j,k) for(int i=(j);i>=(k);i--)
#define pt(a) cerr<<#a<<'='<<a<<' '
#define pts(a) cerr<<#a<<'='<<a<<'\n'
#define int long long
using namespace std;

const int MOD=998244353;

int ksm(int a,int p){
    int ret=1;

    while(p){
        if(p&1) ret=ret*a%MOD;

        a=a*a%MOD;
        p>>=1;
    }

    return ret;
}

// 期望里要做除法, 模意义下就是乘逆元。
int inv(int x){
    return ksm(x,MOD-2);
}

// dp[l][r][m]:
//
// 对当前正在研究的容器 i,
// 左边最近一个比它强的容器离它 l+1 个位置,
// 右边最近一个比它强的容器离它 r+1 个位置;
//
// 换句话说, 左边夹着 l 个打不过 i 的容器,
```

```

// 右边夹着 r 个打不过 i 的容器;
//
// m 表示这两个“危险端点”以外,
// 还能够发生多少次和 i 完全没关系的消除。
//
// bd 则是只有一边存在更强容器时对应的二维状态。
int dp[55][55][55],a[55],bd[55][55];

// L[i],R[i] 分别是 i 左右最近的比 a[i] 大的位置。
// stk 是下面单调栈用的。
int L[55],R[55],stk[55];

signed main()
{
    ios::sync_with_stdio(0);
    cin.tie(0);
    cout.tie(0);

    int n;
    cin>>n;

    rep(i,1,n) cin>>a[i];

    // l=r=m=0 时,
    // 左右两个更强的容器都已经紧贴 i,
    // i 下一轮一旦涉及自己就会直接寄,因此这里作为递推起点放 0。
    dp[0][0][0]=0;//显然如果没有一个位置有剩余,那么当前这个点必然下一刻被打败

    rep(l,0,n) rep(r,0,n) rep(m,0,n){
        if(l+r+m==0) continue;

        // 此时一共有:
        //
        // 左侧 l+1 个“可发生的位置”,
        // 右侧 r+1 个,
        // 外部 m 个。
        //
        // 总数就是 l+1+r+1+m。
        //
        // 如果这一轮发生在左侧,并且 l>0,
        // 那么左边会少掉一个挡在 i 和强者之间的弱容器。
        // 当前这一轮 i 没死,所以还要在后续期望上 +1。
        if(l)
            (dp[l][r][m]+=
             (dp[l-1][r][m]+1)
             *(l+1)%MOD
             *inv(l+1+r+1+m)%MOD
            )%=MOD;

        // 右边完全对称:
        // 消掉右边一个弱容器,状态从 r 变成 r-1。
        if(r)
            (dp[l][r][m]+=
             (dp[l][r-1][m]+1)

```

```

        *(r+1)%MOD
        *inv(l+1+r+1+m)%MOD
    )%=MOD;

    // 如果这一轮发生在外部，
    // 左右缓冲层都没变，只是 m 少了一次。
    if(m)
        (dp[l][r][m]+=
            (dp[l][r][m-1]+1)
            *m%MOD
            *inv(l+1+r+1+m)%MOD
        )%=MOD;

    // 这里故意没有写 l=0 或 r=0 时“危险边”的转移。
    // 因为一旦更强容器已经贴到 i 身边，
    // 那一对被选中时 i 当场被击倒，之后没有存活轮数贡献，
    // 对期望的这一项就是 0，直接不用加。
}

// bd[i][j] 是只有一侧存在更强者时的版本。
// i 表示危险一侧还有多少弱容器挡着，
// j 表示其他地方还可以白白消掉多少次。
bd[0][0]=0;

rep(i,0,n) rep(j,0,n){
    if(i+j==0) continue;

    // 危险一侧还有 i 个弱容器时，
    // 这一侧有 i+1 个可以被选中的相邻位置。
    if(i)
        (bd[i][j]+=
            (bd[i-1][j]+1)
            *(i+1)%MOD
            *inv(i+1+j)%MOD
        )%=MOD;

    // 决斗发生在别的地方，
    // 当前容器再活一轮，同时 j--。
    if(j)
        (bd[i][j]+=
            (bd[i][j-1]+1)
            *j%MOD
            *inv(i+1+j)%MOD
        )%=MOD;
}

int top=0;

// 单调栈求每个位置左边最近的、比自己强的容器。
rep(i,1,n){
    // 栈顶如果比当前 a[i] 还小，
    // 它已经不可能成为 i 左边最近的更强者，直接弹掉。
    while(top&&a[stk[top]]<a[i]) top--;

```

```

        if(!top)
            // 栈空说明左边根本没有比 i 强的。
            L[i]=-1;
        else
            // 否则当前栈顶就是最近的那个更强者。
            L[i]=stk[top];

        // 当前点入栈，给后面的位置当候选。
        stk[++top]=i;
    }

    top=0;

    // 从右往左再跑一次，同理求 R[i]。
    per(i,n,1){
        while(top&&a[stk[top]]<a[i]) top--;

        if(!top)
            R[i]=-1;
        else
            R[i]=stk[top];

        stk[++top]=i;
    }

    rep(i,1,n){
        if(L[i]==-1&&R[i]==-1)
            // 两边都没有更强的，
            // 由于强度互不相同，i 就是全局最大值。
            // 它一定活到最后，所以存活轮数恒为 n-1。
            cout<<n-1<<' ';

        else if(L[i]==-1)
            // 只有右边有更强者。
            //
            // R[i]-i-1:
            // i 与这个更强者之间夹着的弱容器个数。
            //
            // n-1-R[i]+i:
            // 其余区域总共还能发生多少次无关消除。
            cout<<bd[R[i]-i-1][n-1-R[i]+i]<<' ';

        else if(R[i]==-1)
            // 只有左边有更强者，和上面完全对称。
            cout<<bd[i-L[i]-1][n-1-i+L[i]]<<' ';

        else
            // 两边都有更强者，直接扔进三维 dp。
            //
            // R[i]-i-1 是右边缓冲层长度，
            // i-L[i]-1 是左边缓冲层长度，
            // n-1-R[i]+L[i] 是整个 [L,R] 外能够发生的无关消除次数。
            cout<<dp[R[i]-i-1][i-L[i]-1][n-1-R[i]+L[i]]<<' ';
    }
}

```

```
    return 0;
}
```

P6042 「ACOI2020」学园祭

原题目链接: <https://www.luogu.com.cn/problem/P6042>

题解链接: [洛谷题解](#)

代码解释

```
#include<bits/stdc++.h>
using namespace std;
const int N=1e6+5,mod=10086001;
// 题解把答案化成  $\sum a!*(n-2a)^2$ ,  $a$  的上界是  $\text{floor}((n-1)/2)$ 。
// 三个前缀和分别对应展开后的  $n^2$ 、 $-4an$ 、 $4a^2$  三项。
long long frc[N],pre1[N],pre2[N],pre3[N],ans[N],n,t,q,u;
int main(){
    frc[0]=pre1[0]=1;
    // 只需预处理到  $N/2$ , 因为询问中的  $a$  不会更大。
    for(int i=1;i<N>>1;i++){
        frc[i]=(frc[i-1]*i)%mod;
        pre1[i]=(pre1[i-1]+frc[i])%mod;
        pre2[i]=(pre2[i-1]+frc[i]*4*i)%mod;
        pre3[i]=(pre3[i-1]+frc[i]*4*i%mod*i)%mod;
    }
    scanf("%lld",&t);
    while(t--){
        scanf("%lld",&q);
        //  $a$  的范围只到  $\text{floor}((q-1)/2)$ , 三次前缀和让单次询问  $O(1)$ 。
        u=(q-1)/2;
        printf("%lld\n",((pre1[u]*q%mod*q-pre2[u]*q+pre3[u])%mod+mod)%mod);
    }
    return 0;
}
```

P4977 毒瘤之神异之旅

原题目链接: <https://www.luogu.com.cn/problem/P4977>

题解链接: [空气粽子-Air《题解: P4977 毒瘤之神异之旅》](#)

代码解释

```
#include<bits/stdc++.h>
using namespace std;

#define int long long

const int MAXN=10010,mod=1e9+7;

// cheng[i] 预处理  $i^m$ ,
// 因为能力值为  $i$  的守卫对总战力贡献就是  $i^m$ 。
int n,m,k,cheng[MAXN];

// 题解原本想开二维 dp, 后来发现会爆空间。
// 这里每一轮动态申请一行, 算完上一行就释放,
// 本质上还是滚动数组。
int *dp[MAXN]; // 滚动数组

int ksm(int a,int b)
{
    int res=1;

    // 快速幂算  $a^b$ 。
    while(b)
    {
        if(b&1)
            res=res*a%mod;

        a=a*a%mod;
        b>>=1;
    }

    return res;
}

signed main( )
{
    cin>>n>>k>>m;

    // dp[0] 表示还没放任何守卫时的这一层。
    // 多开 k 个位置只是为了保险。
    dp[0]=new int[n+k+1];

    // 每种可能的能力值对应的  $j^M$  会用很多次,
    // 提前算掉, 后面统计答案就不用重复快速幂。
    for(int i=1;i<=n;i++)
        cheng[i]=ksm(i,m); // 预处理。

    // 初始整行清零。
    for(int i=0;i<=n;i++)
        dp[0][i]=0;

    // 什么都没分、总和为 0, 只有一种空方案。
    dp[0][0]=1;
```

```
int ans=0;

// i 表示现在考虑到第 i 层 / 第 i 个有序后的守卫。
for(int i=1;i<=k;i++)
{
    // 新开当前这一层。
    // 它只依赖上一层和当前层已经算过的位置，
    // 所以上一层根本没必要留。
    dp[i]=new int[n+k+1]; //传统的滚动数组没想出来，于是用了这种奇奇怪怪的写法。

    // 新申请的内存没有默认清零，手动清。
    for(int j=0;j<=n;j++)
        dp[i][j]=0;

    // 把 i 划分成 i 个正整数只能全是 1，
    // 所以这个最小状态有且只有一种方案。
    dp[i][i]=1;

    for(int j=i;j<=n;j++)
        // 经典整数划分转移：
        //
        // dp[i-1][j-1]:
        // 当前划分里至少有一个 1，
        // 拿掉这个 1 后就是把 j-1 划成 i-1 份。
        //
        // dp[i][j-i]:
        // 当前 i 个数全部至少为 2，
        // 每个数都减 1，总和正好少掉 i，
        // 于是对应把 j-i 划成 i 份。
        //
        // 两类情况互不相交，直接相加。
        dp[i][j]=(dp[i-1][j-1]+dp[i][j-i])%mod;

    for(int j=1;j<=n;j++)
    {
        int num=0;

        // 假设 a 已经按非降序排列。
        //
        // 当前第 i 层选到了能力值 j 后，
        // 后面的 k-i+1 个位置都不能比 j 小。
        //
        // 所以它们至少要消耗：
        // j*(k-i+1)
        //
        // 如果这一坨已经超过 n，当前 j 显然不合法。
        if(j*(k-i+1)<=n)
            // 先把后面这些位置每个都固定贡献 j，
            // 剩下：
            //
            // n-j*(k-i+1)
            //
            // 再交给前 i-1 层去分。
```

```
        //
        // dp[i-1][...] 就是在数这样的划分方案。
        num+=dp[i-1][n-j*(k-i+1)]; //统计答案。

        // 每一种这样的方案里，
        // 当前这个能力值 j 都会贡献 j^m，
        // cheng[j] 已经提前算好了。
        ans=(ans+1ll*num*cheng[j]%mod)%mod;
    }

    // 当前 i 层算完以后，
    // 下一轮只需要 dp[i]，
    // 所以 dp[i-1] 可以直接释放。
    //
    // 这样空间从 O(nk) 压到了 O(n) 量级。
    delete []dp[i-1];

    // 释放后将指针置空，
    // 防止后面误用悬空指针。
    dp[i-1]=0;
}

cout<<ans;
}
```